

TRƯỜNG ĐẠI HỌC BÁCH KHOA TP. HỒ CHÍ MINH
KHOA KHOA HỌC ỨNG DỤNG
BỘ MÔN TOÁN ỨNG DỤNG



BÀI TẬP LỚN
XÁC SUẤT THỐNG KÊ (MT2013)

Computer Parts (CPUs and GPUs)

Giảng viên hướng dẫn: ThS. Nguyễn Kiều Dung
Nhóm: MT16 - HK232

STT	Họ và tên	MSSV	Lớp	Ngành học
1	Trần Đăng Bảo	2210270	L06	Khoa học máy tính
2	Lê Hoàng Kiệt	2211753	L03	Khoa học máy tính
3	Nguyễn Đình Bằng	2210298	L03	Khoa học máy tính
4	Đặng Quốc Bảo	2210200	L03	Khoa học máy tính
5	Nguyễn Ngọc Châu Phúc	2212631	L04	Khoa học máy tính

Thành phố Hồ Chí Minh, tháng 3 năm 2024

BÁO CÁO KẾT QUẢ LÀM VIỆC NHÓM

STT	Họ và tên	MSSV	Nhiệm vụ	Đóng góp
1	Trần Đăng Bảo	2210270		100%
2	Lê Hoàng Kiệt	2211753		100%
3	Nguyễn Đình Bằng	2210298		100%
4	Đặng Quốc Bảo	2210200		100%
5	Nguyễn Ngọc Châu Phúc	2212631		100%

Họ và tên nhóm trưởng: **Trần Đăng Bảo**

Số ĐT: **0961501846**

Email: **bao.trandang@hcmut.edu.vn**

Nhận xét của GV:

.....

Giảng viên (Ký và ghi rõ họ, tên)	Nhóm trưởng (Ký và ghi rõ họ, tên)
ThS. Nguyễn Kiều Dung	

TÀI LIỆU SƯU TẬP
BỞI HCMUT-CNCP

Mục lục

Danh sách bảng	1
Danh sách hình vẽ	2
1 Tổng quan dữ liệu	3
2 Kiến thức nền	5
2.1 Lý thuyết mẫu	5
2.1.1 Các khái niệm	5
2.1.2 Các đặc trưng của mẫu	5
2.2 Lí thuyết ước lượng	6
2.2.1 Các khái niệm	6
2.2.2 Ước lượng khoảng tin cậy của tổng thể	8
2.3 Lí thuyết kiểm định	9
2.3.1 Khái niệm chung về kiểm định	9
2.3.2 Kiểm định giá trị trung bình μ của tổng thể	10
2.4 Phân tích phương sai (Analysis of Variance - ANOVA)	12
2.5 Hồi quy tuyến tính bội	14
2.5.1 Khái niệm	14
2.5.2 Hệ số xác định (R-square)	15
3 Tiền xử lý số liệu	17
3.1 Đọc dữ liệu	17
3.2 Chọn lọc dữ liệu	17
3.3 Xử lí định dạng dữ liệu	18
3.4 Xử lí dữ liệu khuyết	22
4 Thống kê mô tả	23
4.1 Các giá trị đặc trưng của mẫu	23
4.2 Các loại đồ thị	24
4.3 Mối liên hệ giữa các biến	25
5 Thống kê suy diễn	27
5.1 Tìm khoảng tin cậy một mẫu	27
5.1.1 Mục tiêu	27
5.1.2 Bài toán	27
5.1.3 Các lệnh R sử dụng	27
5.1.4 Tiền xử lý bài toán	27
5.1.5 Giải bài toán trên R	27
5.1.6 Nhận xét	29
5.1.7 Kết luận	29
5.2 Kiểm định hai mẫu	29
5.2.1 Mục tiêu	29

5.2.2	Bài toán	29
5.2.3	Các lệnh R sử dụng	29
5.2.4	Tiền xử lý bài toán	29
5.2.5	Giải quyết bài toán trên R	31
5.2.6	Nhận xét	33
5.2.7	Kết luận	33
5.3	Phân tích phương sai	33
5.3.1	Mục tiêu	33
5.3.2	Bài toán	33
5.3.3	Điều kiện để thực hiện bài toán Anova	33
5.3.4	Phân tích phương sai	38
5.3.5	Kết luận	40
5.4	Hồi quy tuyến tính	42
5.4.1	Mục tiêu	42
5.4.2	Bài toán	42
5.4.3	Xây dựng mô hình hồi quy tuyến tính	42
5.4.4	Kết luận	49
6	Thảo luận và mở rộng	50
7	Nguồn dữ liệu và nguồn code	51
	Tài liệu tham khảo	52

TÀI LIỆU SƯU TẬP
BỞI HCMUT-CNCP



Danh sách bảng

5.1	Các lệnh R sử dụng trong bài toán	27
5.2	Các lệnh R sử dụng trong bài toán	29





Danh sách hình vẽ

2.1	Biểu đồ số giờ xem TV trung bình mỗi tuần của người Anh (UK) và người Mỹ (USA)	7
2.2	Bảng tra hàm phân phối chuẩn	8
2.3	2 vùng đổ tương ứng với diện tích 2 phần $z_{\alpha/2}$	9
2.4	Bảng tra phân vị hàm t-Student	9
2.5	Hình minh họa miền chấp nhận và miền bác bỏ giả thuyết H_0	10
2.6	Hình minh họa miền chấp nhận và miền bác bỏ giả thuyết H_0	10
2.7	Hình minh họa xác suất mắc sai lầm loại I và loại II, với	11
2.8	Phân phối của thống kê chuẩn hoá và vùng bác bỏ	11
2.9	Bảng tra hàm F-distribution ứng với $\alpha = 0,05$	14
2.10	Sự khác biệt khi R^2 cao và thấp	16
3.1	Một vài dòng đầu tiên của dữ liệu	17
3.2	Vài phần tử của dữ liệu sau khi chọn lọc	18
3.3	Vài phần tử đầu của dữ liệu sau khi định dạng	18
3.4	Kết quả sau khi tách cột	20
3.5	Số lượng các đơn vị	20
3.6	Kết quả sau khi đồng nhất đơn vị	21
3.7	Kết quả kiểm tra dữ liệu	21
3.8	Tỉ lệ bị khuyết của dữ liệu	22
3.9	Kết quả kiểm tra khuyết	22
4.1	Các giá trị đặc trưng thu được	23
4.2	Các giá trị đặc trưng của Release_Price	23
4.3	Biểu đồ phân phối tần số một số thông số của mẫu	24
4.4	Ma trận tán xạ tổng hợp thể hiện mối liên hệ giữa các mẫu	26
5.1	Biểu đồ QQ-plot cho mẫu của Nvidia và AMD	30

1 Tổng quan dữ liệu

Tập tin **All_GPUs.csv** là một tập dữ liệu của hơn 3400 GPU (Graphic Processing Unit - một linh kiện chuyên dụng được thiết kế để xử lý nhanh các tác vụ liên quan đến đồ họa trên máy tính) từ Nvidia, AMD và Intel cũng như các thông số kỹ thuật của bộ phận này. Trong hơn 30 thông số về GPU được mô tả trong tập tin, dưới đây là một vài thông số đáng chú ý:

- **Best Resolution:** Độ phân giải tối ưu nhất mà GPU có thể xử lý để hiển thị hình ảnh chất lượng cao trên màn hình. Độ phân giải được đo bằng số điểm ảnh trên chiều rộng và chiều cao của màn hình, ví dụ: 1920x1080 (Full HD) hoặc 3840x2160 (4K Ultra HD). Độ phân giải cao hơn ngoài giới hạn này có thể gây ra hiện tượng giật, chậm hoặc không ổn định.
- **Boost Clock:** Tần số tối đa mà GPU có thể hoạt động khi cần đạt hiệu năng cao. Thông thường, GPU sẽ hoạt động ở một tần số cơ bản (Base Clock), nhưng khi cần nhiều sức mạnh hơn (ví dụ như chơi game hoặc xử lý các video độ phân giải cao, đòi hỏi đồ họa phức tạp), Boost Clock sẽ kích hoạt để tăng tốc GPU và cải thiện hiệu suất.
- **Core Speed:** Đây là tốc độ xử lý của GPU, được đo bằng MHz hoặc GHz, thể hiện tốc độ làm việc của các nhân xử lý trung tâm (core) trong GPU. Tốc độ này ảnh hưởng trực tiếp đến hiệu năng xử lý đồ họa của GPU.
- **Max Power:** Đây là công suất tối đa mà GPU có thể tiêu thụ khi hoạt động ở hiệu năng cao nhất. Đơn vị được sử dụng là watt (W). Thông thường, các GPU mạnh hơn thường tiêu thụ nhiều năng lượng hơn.
- **Memory:** Đây là bộ nhớ (VRAM - Video Random Access Memory) được sử dụng bởi GPU để lưu trữ dữ liệu đồ họa và thông tin cần thiết cho quá trình xử lý đồ họa. Bộ nhớ này có thể là GDDR6, GDDR6X hoặc các loại khác, và được đo bằng đơn vị gigabyte (GB). Độ lớn của bộ nhớ này ảnh hưởng đến khả năng GPU xử lý các tác vụ đồ họa phức tạp.
- **Memory Bandwidth:** Băng thông bộ nhớ, thể hiện tốc độ tối đa mà GPU có thể truyền qua VRAM trong một đơn vị thời gian. Đơn vị được sử dụng là gigabyte/giây (GB/s). Băng thông bộ nhớ càng cao, GPU càng có khả năng truyền dữ liệu nhanh hơn và cải thiện hiệu suất xử lý đồ họa.
- **Memory Bus:** Đây là kênh truyền dẫn dữ liệu giữa GPU và bộ nhớ VRAM. Độ rộng của Memory Bus ảnh hưởng đến khả năng truyền dữ liệu giữa GPU và bộ nhớ VRAM. Memory Bus thường được đo bằng bit, ví dụ 256-bit hoặc 384-bit.
- **Memory Speed:** Đây là tốc độ hoạt động mà GPU có thể truy xuất hoặc đọc dữ liệu từ bộ nhớ VRAM, được đo bằng MHz hoặc GHz.
- **PSU (Power Supply Unit):** Đây là thiết bị cung cấp năng lượng cho GPU và các linh kiện khác trong hệ thống máy tính. PSU được tính theo công suất (watt) và cần đủ công suất để cung cấp cho toàn bộ hệ thống máy tính, bao gồm cả GPU.
- **Pixel Rate:** Đây là số lượng pixel (điểm ảnh) mà GPU có thể xử lý trong một giây. Pixel Rate được đo bằng đơn vị gigapixel/giây (GPixel/s) hoặc terapixel/giây (TPixel/s).

- **Process:** Đây là quy trình sản xuất và công nghệ chế tạo GPU. Quy trình này thường được đo bằng đơn vị nanometer (nm) và thể hiện kích thước transistor trên chip GPU.
- **ROPs (Raster Operations Pipelines):** Đường ống xử lý trên GPU có nhiệm vụ xử lý các phép toán liên quan đến rasterization (quá trình chuyển đổi đồ họa 3D thành hình ảnh 2D trên màn hình).
- **Resolution WxH:** Đây là kích thước độ phân giải của màn hình hoặc hình ảnh. Được đo bằng số điểm ảnh theo chiều rộng (W) và chiều cao (H).
- **TMUs (Texture Mapping Units):** Đây là các đơn vị trong GPU có nhiệm vụ xử lý các phép toán liên quan đến ánh xạ và chuyển đổi các hình ảnh hoặc vật liệu thành dữ liệu được sử dụng trong quá trình làm mịn và hiển thị các đối tượng 3D trên màn hình.
- **Texture Rate:** Đây là số lượng dữ liệu texturing (ánh xạ) mà GPU có thể xử lý trong một giây. Texture Rate được đo bằng đơn vị gigatexels/giây (GTexels/s) hoặc teratexels/giây (TTexels/s).

Bên cạnh đó, tập tin còn cung cấp các thông tin khác về GPU như kiến trúc (**Architecture**), dung lượng bộ nhớ cache cấp 2 (**L2 Cache**), tên nhà sản xuất (**Manufacturer**), ngày ra mắt (**Release date**), giá bán chính thức khi phát hành lần đầu (**Release price**), cũng như số lượng các cổng xuất hình ảnh như **Displayport**, **HDMI**, **VGA** và khả năng hỗ trợ cầu nối **SLI (Scalable Link Interface)** và **Crossfire** (công nghệ cho phép GPU làm việc song song và chia sẻ công việc xử lý qua đó tăng hiệu suất)...

TÀI LIỆU SƯU TẬP
BỞI HCMUT-CNCP

2 Kiến thức nền

2.1 Lý thuyết mẫu

2.1.1 Các khái niệm

Tổng thể: (population) là tập hợp hoàn chỉnh của tất cả các đơn vị hoặc cá thể mà người nghiên cứu muốn thu thập, quan sát. Tổng thể chứa toàn bộ dữ liệu có thể về một đối tượng nghiên cứu nhất định, có thể là một nhóm dân số, một tập hợp các sản phẩm, một tập hợp các văn bản, hoặc bất kỳ thực thể nào khác mà nghiên cứu muốn xem xét. Ví dụ, nếu quan tâm đến chiều cao của tất cả học sinh trong một trường học, thì tổng thể sẽ là tất cả các học sinh trong trường.

Mẫu: (sample) là một phần của tổng thể được lựa chọn để nghiên cứu. Mục tiêu của việc chọn một mẫu là để thu thập dữ liệu có thể phản ánh hoặc đại diện cho toàn bộ tổng thể mà không cần phải khảo sát toàn bộ cá nhân trong tổng thể đó. Điều này giúp giảm chi phí và thời gian cần thiết cho nghiên cứu. Mẫu cần được lựa chọn một cách ngẫu nhiên và đại diện cho tổng thể để kết quả của nghiên cứu có thể được áp dụng cho tổng thể. Tiếp tục ví dụ trên, nếu chỉ chọn một số học sinh từ trường để đo chiều cao, thì đó sẽ là mẫu.

Đặc điểm của các thuộc tính dữ liệu Tất cả các đối tượng trong tổng thể và mẫu có thể được mô tả bằng các đặc trưng (hoặc thuộc tính). Đặc trưng có thể được phân loại thành hai loại chính: định tính và định lượng.

- **Đặc trưng định tính (categorical features):** Đây là các đặc trưng mà mỗi đối tượng thuộc vào một nhóm hoặc loại cụ thể. Ví dụ, trong một nghiên cứu về loài chim, một đặc trưng định tính có thể là "loại chim" với các giá trị như "vịt", "gà", "cú", và "ngỗng". Các đặc trưng định tính thường được biểu diễn bằng các nhãn, chuỗi ký tự hoặc số mã hóa.
- **Đặc trưng định lượng (quantitative features):** Đây là các đặc trưng mà có thể được đo lường bằng các giá trị số. Ví dụ, trong một nghiên cứu về chiều cao của con người, đặc trưng định lượng có thể là "chiều cao" và được biểu diễn bằng giá trị số như 160cm, 175cm, 183cm, và 190cm.

2.1.2 Các đặc trưng của mẫu

Trong thống kê và phân tích dữ liệu, các khái niệm như trung bình, phương sai, độ lệch chuẩn, sai số chuẩn và tỉ lệ phần trăm mang dấu hiệu nghiên cứu là các công cụ quan trọng giúp ta hiểu và diễn giải dữ liệu.

- **Kích thước mẫu (n):** Kích thước mẫu (sample size) là số lượng các quan sát, thí nghiệm hoặc nguyên mẫu được thu thập hoặc sử dụng trong một nghiên cứu hoặc phân tích dữ liệu. Nó đại diện cho số lượng các trường hợp, đối tượng hoặc điểm dữ liệu mà các phép đo, quan sát hoặc thử nghiệm được thực hiện.
- **Trung bình:** Đây là giá trị đại diện trung bình của tập hợp số liệu, được tính bằng tổng của tất cả các giá trị trong tập hợp chia cho số lượng giá trị. Trung bình phản ánh vị trí trung tâm của tập hợp dữ liệu (giá trị trung tâm của tập dữ liệu) và thường được sử dụng như một đại diện cho toàn bộ tập dữ liệu.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

- **Phương sai mẫu:** Đây là một thước đo biến thiên hoặc phân tán của tập hợp số liệu so với trung bình của chúng. Phương sai được tính bằng cách lấy trung bình của bình phương khoảng cách từ mỗi điểm dữ liệu đến trung bình. Một phương sai cao chỉ ra sự phân tán lớn của dữ liệu so với trung bình, và ngược lại.

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (2)$$

- **Độ lệch chuẩn mẫu:** Đây là căn bậc hai của phương sai, đo lường độ phân tán của dữ liệu so với trung bình. Độ lệch chuẩn được sử dụng phổ biến vì nó đặt dữ liệu và độ phân tán của chúng trên cùng một thang đo với dữ liệu gốc, làm cho việc diễn giải dễ dàng hơn.

$$s = \sqrt{s^2} \quad (3)$$

- **Sai số chuẩn (Standard Error):** Đây là ước lượng về mức độ không chính xác mà thông qua đó trung bình mẫu có thể phản ánh trung bình của tổng thể. Nó được tính từ độ lệch chuẩn của dữ liệu chia cho căn bậc hai của kích thước mẫu. Sai số chuẩn thường được sử dụng trong các bài báo khoa học để đánh giá mức độ tin tưởng vào trung bình mẫu.

$$SE(\bar{x}) = \frac{s}{\sqrt{n}} \quad (4)$$

- **Tỉ lệ phần tử có tính chất A:** Được sử dụng để mô tả tỉ lệ người hoặc đối tượng trong một nhóm cụ thể mang một dấu hiệu nghiên cứu nhất định, ví dụ như tỉ lệ người mắc một loại bệnh cụ thể trong một quần thể. Tỉ lệ này giúp nhận diện mức độ phổ biến của dấu hiệu hoặc biến số cần quan tâm trong quần thể nghiên cứu.

$$f = \frac{m}{n} \quad (5)$$

Với m đại diện cho số lượng các phần tử trong mẫu (sample) mang dấu hiệu, tính chất hoặc biến số mà nghiên cứu quan tâm và n là kích thước mẫu.

2.2 Lí thuyết ước lượng

2.2.1 Các khái niệm

Lí thuyết ước lượng là một phần quan trọng trong lĩnh vực thống kê và nghiên cứu khoa học, chủ yếu liên quan đến việc tìm cách xác định giá trị của các tham số (parameters) của một quần thể dựa trên các mẫu (samples) được rút ra từ quần thể đó. Thông thường ta cần ước lượng trung bình tổng thể (, phương sai tổng thể (và tỉ lệ các phần tử có tính chất nào đó trong tổng thể (

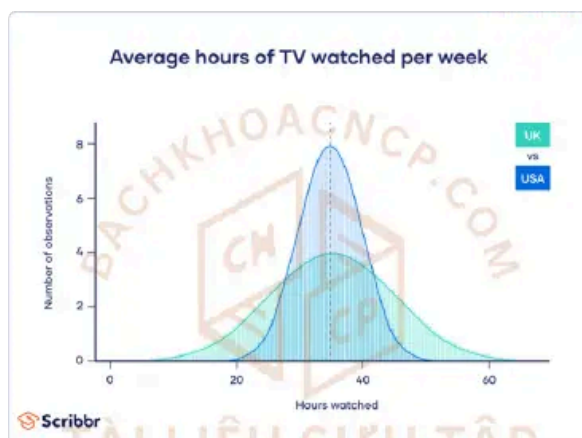
- **Khoảng Tin Cậy (Confidence Interval - CI):** Là một loại ước lượng khoảng được sử dụng để chỉ ra phạm vi mà ta tin rằng tham số của tổng thể nằm trong đó. Khoảng tin cậy thường được xác định bởi hai giới hạn: giới hạn dưới và giới hạn trên. Ví dụ, một khoảng tin cậy 95% cho trung bình tổng thể có thể là (20, 30), nghĩa là ta tin rằng với độ tin cậy 95%, trung bình thực sự của tổng thể nằm trong khoảng từ 20 đến 30.
- **Mức Ý Nghĩa (Significance Level - α):** Là ngưỡng mà ta chọn để quyết định ý nghĩa. Theo

quy tắc chung, mức ý nghĩa thường được chọn ở mức 0,05 - nghĩa là khả năng kết quả quan sát sự khác biệt được nhìn thấy trên số liệu là ngẫu nhiên chỉ là 5%.

- **Độ Tin Cậy (Confidence Level):** Được biểu thị dưới dạng một tỷ lệ phần trăm chỉ mức độ tin tưởng hoặc sự chắc chắn mà khoảng tin cậy ước lượng của chúng ta bao gồm tham số tổng thể thực sự. Ví dụ: nếu ta xây dựng khoảng tin cậy với mức tin cậy 95%, ta tin chắc rằng 95 trên 100 lần ước tính sẽ nằm giữa giá trị trên và giá trị dưới được chỉ định bởi khoảng tin cậy.

$$\text{Confidence level} = 1 - \alpha \quad (6)$$

Ví dụ: Khảo sát 100 người Anh và người Mỹ về thói quen xem TV, nhận thấy rằng thời gian trung bình của 2 nhóm người này khi xem TV đều là 35 giờ/tuần. Tuy nhiên, ước lượng về thời gian xem TV của người Anh sẽ có một khoảng tin cậy rộng hơn của người Mỹ do có sự biến động dữ liệu lớn hơn.



Hình 2.1. Biểu đồ số giờ xem TV trung bình mỗi tuần của người Anh (UK) và người Mỹ (USA)

Có hai phương pháp ước lượng thường được sử dụng là ước lượng điểm (point estimation) và ước lượng khoảng (interval estimation).

- **Ước lượng điểm (Point Estimation):** Là phương pháp tìm một giá trị duy nhất (một điểm) dùng làm ước lượng tốt nhất cho tham số của quần thể. Kết quả cần được ước lượng được cho bởi 1 số:

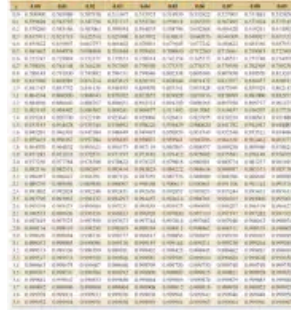
$$\mu \approx \bar{x} \quad (7)$$

$$\sigma^2 \approx s^2 \quad (8)$$

$$p \approx f \quad (9)$$

Ví dụ, giả sử muốn biết chiều cao trung bình của tất cả sinh viên trong một trường đại học, một ước lượng điểm có thể là chiều cao trung bình của một mẫu nhỏ sinh viên từ trường đó.

- **Ước lượng khoảng (Interval Estimation):** Thay vì cung cấp một giá trị duy nhất như ước lượng điểm, ước lượng khoảng cho biết phạm vi giá trị mà tham số có thể nằm trong với một mức độ tin cậy nhất định (ví dụ, 95% hoặc 99% tin cậy). Kết quả ước lượng được cho bởi 1 khoảng (a, b).



Hình 2.2. Bảng tra hàm phân phối chuẩn

$$(\bar{x} - \epsilon, \bar{x} + \epsilon) \quad (10)$$

Ví dụ, thay vì nói chiều cao trung bình của sinh viên là 170 cm, ta có thể nói rằng ta 95% tin tưởng rằng chiều cao trung bình nằm trong khoảng từ 168 cm đến 172 cm.

2.2.2 Ước lượng khoảng tin cậy của tổng thể

Xác định khoảng ước lượng đối xứng của trung bình tổng thể dựa vào một mẫu đã cho, với kích thước là n , trung bình mẫu là $\bar{x}$ và phương sai mẫu là s^2 hoặc phương sai tổng thể là σ^2 . Mục tiêu là xác định một khoảng ước lượng đối xứng xung quanh μ của mẫu với một mức độ tin cậy cụ thể.

Để giải quyết vấn đề này, ta cần đi đến việc xác định epsilon (ϵ) - sai số ước lượng, dựa trên các thông tin đã biết về mẫu. Khoảng tin cậy sẽ được biểu diễn bằng khoảng $\bar{x} \pm \epsilon$. Tùy thuộc vào giả định về phân phối của dữ liệu và các thông tin đã biết về phương sai, cách tính ϵ sẽ thay đổi như sau:

Nếu X tuân theo phân phối chuẩn hoặc phân phối bất kỳ và phương sai tổng thể, σ^2 , đã biết:

$$\epsilon = z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$

Nếu X tuân theo phân phối chuẩn và phương sai tổng thể, σ^2 , chưa biết:

$$\epsilon = t_{\alpha/2}^{(n-1)} \frac{s}{\sqrt{n}}$$

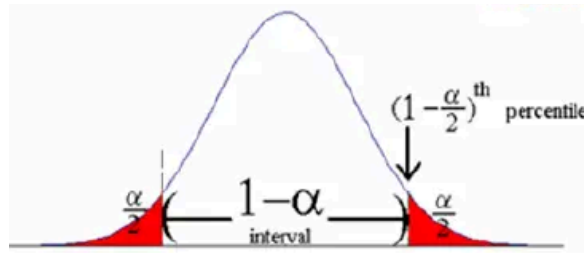
Nếu X tuân theo phân phối bất kỳ và phương sai tổng thể, σ^2 , chưa biết:

$$\epsilon = z_{\alpha/2} \frac{s}{\sqrt{n}}$$

Giá trị $z_{\alpha/2}$ có thể được tìm trong bảng phân phối chuẩn, giúp xác định các giá trị cần thiết để bao phủ một tỷ lệ diện tích nhất định trên đường cong phân phối chuẩn.

Đối với trường hợp không biết phương sai tổng thể (σ^2), ta cần dùng giá trị $t_{\alpha/2}^{(n-1)}$, giá trị này có thể tra từ bảng phân vị t-Student, tương ứng với $\alpha/2$ và số độ tự do là $n - 1$.

Từ việc xác định sai số ước lượng ϵ , khoảng tin cậy trung bình của tổng thể sẽ được biểu diễn thông



Hình 2.3. 2 vùng đỏ tương ứng với diện tích 2 phần $z_{\alpha/2}$

α	0.05	0.025	0.01	0.005	0.001	0.0005	0.0001
1	1.645	1.960	2.576	2.807	3.291	3.481	3.893
2	1.701	2.054	2.689	2.922	3.401	3.592	4.009
3	1.751	2.107	2.719	2.953	3.438	3.625	4.041
4	1.781	2.131	2.744	2.977	3.462	3.648	4.061
5	1.801	2.149	2.764	2.994	3.479	3.663	4.073
6	1.815	2.161	2.776	3.005	3.490	3.672	4.082
7	1.825	2.171	2.786	3.015	3.500	3.680	4.090
8	1.833	2.179	2.794	3.023	3.508	3.687	4.096
9	1.841	2.186	2.801	3.030	3.515	3.693	4.101
10	1.847	2.191	2.806	3.034	3.519	3.696	4.104
11	1.852	2.195	2.810	3.037	3.522	3.699	4.107
12	1.856	2.198	2.813	3.040	3.524	3.701	4.109
13	1.859	2.201	2.815	3.042	3.526	3.703	4.111
14	1.861	2.203	2.817	3.044	3.528	3.705	4.112
15	1.863	2.205	2.819	3.046	3.529	3.706	4.113
16	1.865	2.207	2.820	3.047	3.531	3.708	4.114
17	1.867	2.209	2.821	3.048	3.532	3.709	4.115
18	1.868	2.210	2.822	3.049	3.533	3.710	4.116
19	1.869	2.211	2.823	3.050	3.534	3.711	4.117
20	1.870	2.212	2.824	3.051	3.535	3.712	4.118
21	1.871	2.213	2.825	3.052	3.536	3.713	4.119
22	1.872	2.214	2.825	3.053	3.537	3.714	4.120
23	1.873	2.215	2.826	3.054	3.538	3.715	4.121
24	1.874	2.216	2.827	3.055	3.539	3.716	4.122
25	1.875	2.217	2.828	3.056	3.540	3.717	4.123
26	1.876	2.218	2.829	3.057	3.541	3.718	4.124
27	1.877	2.219	2.830	3.058	3.542	3.719	4.125
28	1.878	2.220	2.831	3.059	3.543	3.720	4.126
29	1.879	2.221	2.832	3.060	3.544	3.721	4.127
30	1.880	2.222	2.833	3.061	3.545	3.722	4.128
31	1.881	2.223	2.834	3.062	3.546	3.723	4.129
32	1.882	2.224	2.835	3.063	3.547	3.724	4.130
33	1.883	2.225	2.836	3.064	3.548	3.725	4.131
34	1.884	2.226	2.837	3.065	3.549	3.726	4.132
35	1.885	2.227	2.838	3.066	3.550	3.727	4.133
36	1.886	2.228	2.839	3.067	3.551	3.728	4.134
37	1.887	2.229	2.840	3.068	3.552	3.729	4.135
38	1.888	2.230	2.841	3.069	3.553	3.730	4.136
39	1.889	2.231	2.842	3.070	3.554	3.731	4.137
40	1.890	2.232	2.843	3.071	3.555	3.732	4.138
41	1.891	2.233	2.844	3.072	3.556	3.733	4.139
42	1.892	2.234	2.845	3.073	3.557	3.734	4.140
43	1.893	2.235	2.846	3.074	3.558	3.735	4.141
44	1.894	2.236	2.847	3.075	3.559	3.736	4.142
45	1.895	2.237	2.848	3.076	3.560	3.737	4.143
46	1.896	2.238	2.849	3.077	3.561	3.738	4.144
47	1.897	2.239	2.850	3.078	3.562	3.739	4.145
48	1.898	2.240	2.851	3.079	3.563	3.740	4.146
49	1.899	2.241	2.852	3.080	3.564	3.741	4.147
50	1.900	2.242	2.853	3.081	3.565	3.742	4.148

Hình 2.4. Bảng tra phân vị hàm t-Student

qua khoảng sau:

$$(\bar{x} - \epsilon, \bar{x} + \epsilon) \quad (11)$$

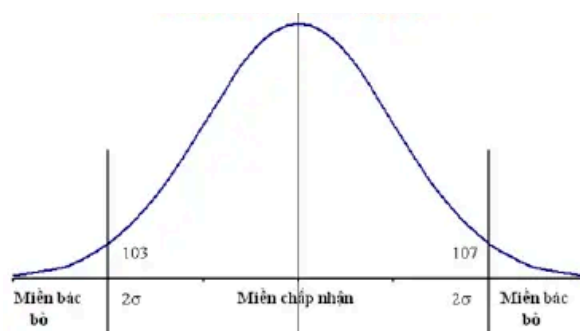
2.3 Lí thuyết kiểm định

2.3.1 Khái niệm chung về kiểm định

Trong thống kê, kiểm định (hypothesis testing) là quá trình đánh giá một giả thuyết về dữ liệu để xác định xem liệu có đủ bằng chứng để chấp nhận hay bác bỏ giả thuyết đó. Mục tiêu của kiểm định là đưa ra quyết định dựa trên dữ liệu mẫu có sẵn để rút ra những kết luận về tổng thể.

Quá trình kiểm định thường bắt đầu bằng việc xây dựng hai giả thuyết: **Giả thuyết không (null hypothesis, ký hiệu H_0)**: Đây là giả thuyết mà chúng ta muốn kiểm tra hoặc bác bỏ. Thông thường, giả thuyết không cho rằng không có sự khác biệt hoặc tác động đáng kể giữa các nhóm hoặc biến số trong tổng thể. **Giả thuyết thay thế (alternative hypothesis, ký hiệu H_1)**: Đây là giả thuyết mà chúng ta muốn chấp nhận nếu có đủ bằng chứng để bác bỏ giả thuyết H_0 . Giả thuyết thay thế thường cho rằng có sự khác biệt hoặc tác động đáng kể giữa các nhóm hoặc biến số trong tổng thể.

Ví dụ: Giả sử bạn là một nhà nghiên cứu y tế quan tâm đến tác động của việc tập thể dục hàng ngày đến sức khỏe tim mạch. Bạn muốn xác định xem liệu việc tập thể dục hàng ngày có ảnh hưởng tích cực đến lượng cholesterol trong máu hay không. **Giả thuyết không (H_0)**: Việc tập thể dục hàng ngày không có ảnh hưởng đến lượng cholesterol trong máu. Điều này có nghĩa là không có sự khác biệt đáng kể về lượng cholesterol trong máu giữa nhóm người tập thể dục hàng ngày và nhóm người không tập thể dục hàng ngày. **Giả thuyết thay thế (H_1)**: Việc tập thể dục hàng ngày có ảnh hưởng tích cực đến lượng cholesterol trong máu. Điều này có nghĩa là có sự khác biệt đáng kể về lượng cholesterol trong máu giữa nhóm người tập thể dục hàng ngày và nhóm người không tập thể dục hàng ngày, với



Hình 2.5. Hình minh họa miền chấp nhận và miền bác bỏ giả thuyết H_0

Ho	THỰC TẾ	
	ĐÚNG	SAI
CHẤP NHẬN	Xác suất quyết định ĐÚNG là $1 - \alpha$ 	Ho SAI nhưng CHẤP NHẬN  Loại II
BÁC BỎ	Ho ĐÚNG nhưng BÁC BỎ  Loại I	Xác suất quyết định ĐÚNG là $1 - \beta$ 

Hình 2.6. Hình minh họa miền chấp nhận và miền bác bỏ giả thuyết H_0 .

lượng cholesterol trong máu của nhóm tập thể dục hàng ngày thấp hơn.

Miền Chấp Nhận (Acceptance Region): Là phạm vi giá trị của thống kê kiểm định mà ở đó, giả thuyết H_0 được chấp nhận. Nói cách khác, không có bằng chứng đủ mạnh để bác bỏ giả thuyết H_0 .

Miền Bác Bỏ (Rejection Region): Là phạm vi giá trị của thống kê kiểm định mà ở đó, giả thuyết H_0 được bác bỏ và giả thuyết đối (H_1) được chấp nhận. Các giá trị trong miền này chỉ ra rằng có sự khác biệt đáng kể so với những gì giả định bởi H_0 .

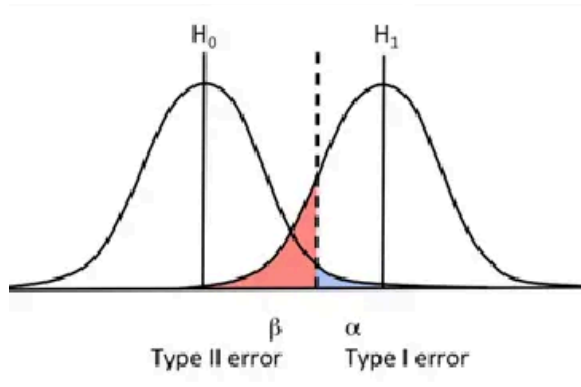
Trong quá trình kiểm định, có thể sẽ xảy ra các loại sai lầm sau đây:

- **Sai lầm loại 1:** Xảy ra khi bác bỏ giả thuyết Null (H_0) trong khi nó thực sự đúng. Tức là, kết luận có sự ảnh hưởng trong khi thực tế không có gì xảy ra.
- **Sai lầm loại 2:** Xảy ra khi không bác bỏ giả thuyết Null (H_0) trong khi nó thực sự sai. Tức là, kết luận không có sự ảnh hưởng trong khi thực tế có sự ảnh hưởng.

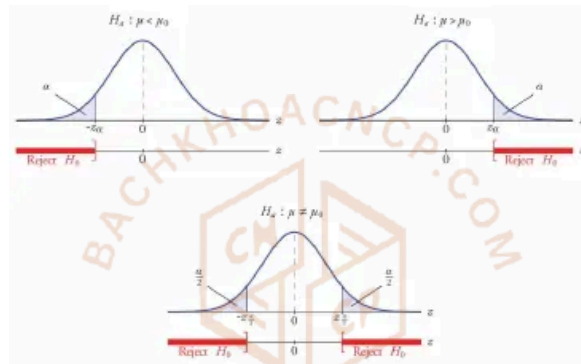
2.3.2 Kiểm định giá trị trung bình μ của tổng thể

Một mẫu có kích thước n , trung bình $\bar{x}$, phương sai s^2 được chọn từ một tổng thể có trung bình μ . Mục tiêu của bài toán kiểm định này là kiểm tra xem với mức ý nghĩa α , liệu giá trị trung bình μ của tổng thể có thực sự đáng tin cậy hay không.

Để giải quyết bài toán này, ta lần lượt xác định các giả thuyết không (H_0) và giả thuyết thay thế (H_1). Thông thường, H_0 sẽ có dạng $\mu = \mu_0$ và tùy theo yêu cầu bài toán mà H_1 có thể là $\mu \neq \mu_0$, $\mu > \mu_0$ hay $\mu < \mu_0$. Sau đó, ta sẽ xác định miền bác bỏ giả thuyết H_0 và tính toán giá trị kiểm định.



Hình 2.7. Hình minh họa xác suất mắc sai lầm loại I và loại II, với α và β lần lượt là tỉ lệ mắc phải 2 lỗi tương ứng



Hình 2.8. Phân phối của thống kê chuẩn hoá và vùng bác bỏ

Nếu giá trị kiểm định thuộc vào miền bác bỏ, ta sẽ bác bỏ giả thuyết H_0 và chấp nhận H_1 , điều này nghĩa là giả thuyết về giá trị trung bình μ của tổng thể không đáng tin cậy với mức ý nghĩa α . Ngược lại, nếu giá trị không nằm trong khu vực từ chối, ta không có đủ bằng chứng để bác bỏ H_0 , tức là giả thuyết về giá trị trung bình của tổng thể vẫn có thể được coi là đáng tin cậy với mức ý nghĩa α đó.

Dựa vào các thông số đã được xác định của tổng thể, ta có thể chia bài toán kiểm định trung bình $\bar{x}$ của tổng thể thành các dạng sau:

Dạng 1: Biết σ tổng thể:

Dạng 2: Chưa biết σ tổng thể và tổng thể có phân phối chuẩn:

Giả thuyết	Tiêu chuẩn kiểm định	Điều kiện bác bỏ H_0
$H_0: \mu = \mu_0$ $H_1: \mu \neq \mu_0$		$ Z > Z_{(\alpha/2)}$
$H_0: \mu = \mu_0$ $H_1: \mu > \mu_0$	$Z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$	$Z > Z_\alpha$
$H_0: \mu = \mu_0$ $H_1: \mu < \mu_0$		$Z < Z_\alpha$

Giả thuyết	Tiêu chuẩn kiểm định	Điều kiện bác bỏ H_0
$H_0: \mu = \mu_0$ $H_1: \mu \neq \mu_0$		$ T > t_{(\alpha/2)}^{(n-1)}$
$H_0: \mu = \mu_0$ $H_1: \mu > \mu_0$	$T = \frac{\bar{x} - \mu}{s\sqrt{n}}$	$T > t_{\alpha}^{n-1}$
$H_0: \mu = \mu_0$ $H_1: \mu < \mu_0$		$T < -t_{\alpha}^{n-1}$

Giả thuyết	Tiêu chuẩn kiểm định	Điều kiện bác bỏ H_0
$H_0: \mu = \mu_0$ $H_1: \mu \neq \mu_0$		$ Z > Z_{(\alpha/2)}$
$H_0: \mu = \mu_0$ $H_1: \mu > \mu_0$	$Z = \frac{\bar{x} - \mu_0}{s\sqrt{n}}$	$Z > Z_{\alpha}$
$H_0: \mu = \mu_0$ $H_1: \mu < \mu_0$		$Z < -Z_{\alpha}$

Dạng 3: Chưa biết σ tổng thể và tổng thể có phân phối bất kì:

2.4 Phân tích phương sai (Analysis of Variance - ANOVA)

ANOVA, viết tắt của "Analysis of Variance," là một kỹ thuật thống kê được sử dụng để so sánh sự khác biệt giữa trung bình của hai hoặc nhiều nhóm dựa trên trung bình của các mẫu quan sát được thông qua kiểm định các giả thuyết về sự bằng nhau của chúng. Phương pháp này giúp xác định liệu có bất kỳ sự khác biệt đáng kể nào giữa các trung bình của các nhóm hay không. **ANOVA một chiều (One-Way ANOVA)** Được sử dụng khi muốn so sánh trung bình giữa hai hoặc nhiều nhóm dựa trên một yếu tố độc lập. **Ví dụ**, phân tích hiệu quả của phương pháp giảng dạy dựa trên giới tính của sinh viên. Trong ví dụ này, yếu tố độc lập duy nhất ảnh hưởng đến trung bình các nhóm là giới tính.

Giả sử ta có n mẫu được chọn từ k tổng thể, mỗi tổng thể có trung bình tổng thể tương ứng là $\mu_1, \mu_2, \dots, \mu_k$. Gọi x_{ij} là lần xem xét thứ j của mẫu thứ i .

Quy trình của ANOVA bao gồm các bước sau:

- **Đưa ra giả thuyết:**

Giả thuyết không H_0 : Không có sự khác biệt đáng kể giữa các nhóm. $H_0: \mu_1 = \mu_2 = \dots = \mu_k$ Giả thuyết đối H_1 : Có ít nhất một cặp nhóm có sự khác biệt đáng kể về trung bình. $H_1: \exists \ i=j \mid \mu_i \neq \mu_j$ với $(i, j \leq k)$

- **Tính toán:** ANOVA tính toán bằng cách phân chia tổng biến độ biến đổi (tổng các bình phương) thành các thành phần có thể giải thích (biến đổi giữa các nhóm) và không thể giải thích (biến đổi trong nhóm). Từ đó, tính ra tỉ lệ F, giá trị này giúp xác định liệu sự khác biệt giữa các nhóm có ý nghĩa hay không.

Cụ thể: Tính giá trị trung bình cho tất cả các mẫu:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^k \sum_{j=1}^{n_i} x_{ij} = \frac{1}{n} \sum_{n=1}^k x_{ij} n_i$$

với $n = \sum_{i=1}^k n_i$, và $\bar{x}_i$ là giá trị trung bình cho từng mẫu:

$$\bar{x}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} x_{ij} \quad (i = 1, 2, \dots, k)$$

Tính SS_i thể hiện sự biến thiên trong từng mẫu:

$$SS_i = \sum_{j=1}^{n_i} (x_{ij} - \bar{x}_i)^2 \quad (i = 1, 2, \dots, k)$$

Tính SSW (tổng các biến thiên trong từng mẫu):

$$SSW = \sum_{i=1}^k \sum_{j=1}^{n_i} (x_{ij} - \bar{x}_i)^2$$

Tính SSB thể hiện sự biến thiên giữa các mẫu:

$$SSB = \sum_{i=1}^k n_i (\bar{x}_i - \bar{x})^2$$

Tính tổng bình phương chênh lệch giữa từng mẫu và trung bình của tất cả các mẫu quan sát:

$$SST = \sum_{i=1}^k \sum_{j=1}^{n_i} (x_{ij} - \bar{x})^2 = SSW + SSB$$

Tính ước lượng phương sai trong nội bộ mẫu (MSW):

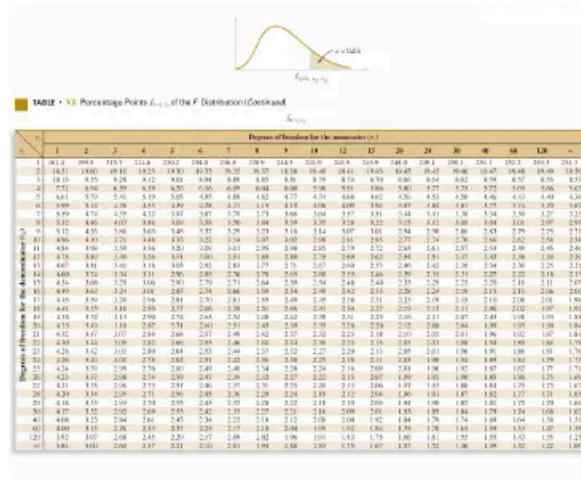
$$MSW = \frac{SSW}{n - k}$$

Tính ước lượng phương sai giữa các mẫu (MSB):

$$MSB = \frac{SSB}{k - 1}$$

Cuối cùng, tính giá trị kiểm định F:

$$F = \frac{MSB}{MSW}$$



Hình 2.9. Bảng tra hàm F-distribution ứng với $\alpha = 0,05$

- **Xác định khoảng bác bỏ và kết luận:** Dựa trên giá trị F và độ tin cậy, chúng ta có thể chấp nhận hoặc bác bỏ giả thuyết H_0 . Miền bác bỏ được xác định như sau:

$$RR = (f_{\alpha}(k - 1, N - k), +\infty)$$

trong đó f_{α} là giá trị F tại mức ý nghĩa α , k là số lượng nhóm, và N là tổng số quan sát.

Nếu giá trị F thuộc khoảng của miền bác bỏ, ta có thể kết luận trung bình của các tổng thể có sự khác nhau và chấp nhận giả thuyết đối H_1 , ngược lại, ta vẫn sẽ chấp nhận H_0 , nghĩa là chưa thể khẳng định được tồn tại sự khác nhau giữa trung bình các tổng thể.

2.5 Hồi quy tuyến tính bội

2.5.1 Khái niệm

Hồi quy tuyến tính là một phương pháp phân tích quan hệ giữa biến phụ thuộc Y với một hay nhiều biến độc lập X . Mô hình hóa sử dụng hàm tuyến tính. Các tham số của mô hình (hay hàm số) được ước lượng từ dữ liệu.

Trong hồi quy tuyến tính đơn, chỉ có một biến độc lập được sử dụng để dự đoán biến phụ thuộc. Tuy nhiên, trong hồi quy tuyến tính bội, có nhiều hơn một biến độc lập được sử dụng.

Mô hình hồi quy tuyến tính bội giả định rằng có một mối quan hệ tuyến tính giữa biến phụ thuộc và các biến độc lập. Mục tiêu của hồi quy tuyến tính bội là tìm ra một phương trình tuyến tính tốt nhất để dự đoán giá trị của biến phụ thuộc dựa trên các giá trị của các biến độc lập.

Phương trình hồi quy tuyến tính bội có dạng:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Trong đó:

- Y là biến phụ thuộc (dependent variable): Đây là biến chúng ta muốn dự đoán hoặc giải thích.
- Trong một mô hình hồi quy tuyến tính bội, chúng ta cố gắng tìm một phương trình tuyến tính

để dự đoán giá trị của biến phụ thuộc Y dựa trên các biến độc lập $X_1, X_2, \dots, X_n$.

- $X_1, X_2, \dots, X_n$ là các biến độc lập (independent variables): Đây là các đại lượng mà chúng ta sử dụng để dự đoán biến phụ thuộc Y . Mỗi biến độc lập X_i ($i = 1$ đến n) có thể là một thuộc tính, một đặc trưng hoặc một biến số mà chúng ta cho là có ảnh hưởng đến biến phụ thuộc.
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ là các hệ số hồi quy (coefficients): Đây là các tham số trong phương trình tuyến tính. Các hệ số hồi quy đại diện cho mức độ ảnh hưởng của từng biến độc lập đến biến phụ thuộc. Hệ số β_0 (cũng được gọi là hệ số điều chỉnh) là hệ số ứng với hạng tự do của mô hình, tức là giá trị dự đoán của biến phụ thuộc khi tất cả các biến độc lập đều bằng 0.
- ϵ là sai số ngẫu nhiên (error term): Đây là thành phần không thể giải thích bằng các biến độc lập trong mô hình. Nó đại diện cho sự biến động tự nhiên và các yếu tố không xác định khác mà có thể ảnh hưởng đến biến phụ thuộc. Sai số ngẫu nhiên ϵ được giả định tuân theo phân phối chuẩn với giá trị trung bình bằng 0.

2.5.2 Hệ số xác định (R-square)

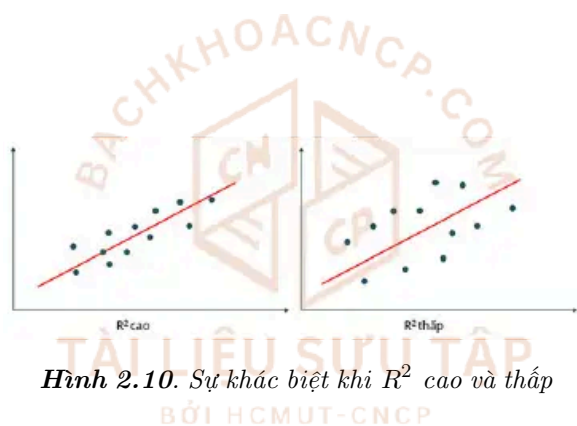
Hệ số xác định (R-square) được biểu diễn bằng công thức toán học sau đây:

$$R^2 = \frac{\text{Sum of Squares Regression}}{\text{Sum of Squares Total}}$$

Trong đó:

- SSR (Sum of Squares Regression): SSR đo lường tổng phương sai giữa dữ liệu quan sát và giá trị dự đoán bởi mô hình hồi quy tuyến tính. Nó đại diện cho phần biệt chênh lệch giữa dữ liệu thực tế và dự đoán của mô hình. SSR thường được sử dụng để đo lường mức độ phù hợp của mô hình hồi quy với dữ liệu.
- SST (Sum of Squares Total): SST đo lường tổng phương sai của dữ liệu quan sát so với giá trị trung bình của dữ liệu. Nó đại diện cho phần biệt chênh lệch giữa dữ liệu thực tế và giá trị trung bình. SST thường được sử dụng để đo lường phạm vi biến thiên của dữ liệu ban đầu.

R-square có ý nghĩa như sau: - R-square = 0: Điều này cho thấy không có sự biến đổi nào trong biến phản hồi có thể được giải thích bởi các biến giải thích trong mô hình. Mô hình không thể dự đoán kết quả. - R-square = 1: Điều này cho thấy toàn bộ sự biến thiên trong biến phản hồi có thể được giải thích bởi các biến giải thích trong mô hình. Mô hình có thể dự đoán kết quả một cách chính xác mà không có sai số từ các biến độc lập. - R-square nằm giữa 0 và 1: Điều này cho thấy một phần của sự biến thiên trong biến phản hồi có thể được giải thích bởi các biến giải thích trong mô hình. Giá trị càng gần 1 thì mô hình càng tốt trong việc giải thích biến đổi của biến phản hồi. Tuy nhiên, R-square có hạn chế vì nó không đánh giá tính chất hay ý nghĩa của các biến độc lập. Nó chỉ đo lường mức độ biến thiên có thể được giải thích bởi mô hình, không xác định xem các biến giải thích nào nên được bao gồm trong mô hình và các biến nào nên loại bỏ. Do đó, việc tăng giá trị R-square bằng cách thêm các biến dự đoán không nhất thiết có nghĩa là mô hình trở nên tốt hơn.



Hình 2.10. Sự khác biệt khi R^2 cao và thấp

3 Tiền xử lý số liệu

3.1 Đọc dữ liệu

Để đọc dữ liệu trong tập tin **All_GPUs.csv**, sử dụng lệnh `read.csv("All_GPUs.csv")`, lưu vào biến `dat`. Lệnh `head(dat)` để hiển thị một số dòng đầu tiên của dữ liệu, mặc định là 6 dòng để xem tổng quát bảng dữ liệu:

```
1 dat<-read.csv(All_GPU.csv)
2 head(dat)
3 View(dat)
```

Source

Console Terminal Background Jobs

R 4.3.3 ~\j

> head(dat)

	Architecture	Best_Resolution	Boost_Clock	Core_Speed	DVI_Connection	Dedicated	Direct_X	
1	Tesla G92b			738 MHz	2	Yes	DX 10.0	
2	R600 XT	1366 x 768		\n-	2	Yes	DX 10	
3	R600 PRO	1366 x 768		\n-	2	Yes	DX 10	
4	RV630	1024 x 768		\n-	2	Yes	DX 10	
5	RV630	1024 x 768		\n-	2	Yes	DX 10	
6	RV630	1024 x 768		\n-	2	Yes	DX 10	
	DisplayPort_Connection	HDMI_Connection	Integrated	L2_Cache	Manufacturer	Max_Power	Memory	
1	NA	0	No	OKB	Nvidia	141 watts	1024 MB	
2	NA	0	No	OKB	AMD	215 watts	512 MB	
3	NA	0	No	OKB	AMD	200 Watts	512 MB	
4	NA	0	No	OKB	AMD	200 Watts	256 MB	
5	NA	0	No	OKB	AMD	45 Watts	256 MB	
6	NA	0	No	OKB	AMD	50 Watts	256 MB	
	Memory_Bandwidth	Memory_Bus	Memory_Speed	Memory_Type			Name	Notebook_GPU
1	64GB/sec	256 Bit	1000 MHz	GDDR3			GeForce GTS 150	No
2	106GB/sec	512 Bit	828 MHz	GDDR3			Radeon HD 2900 XT 512MB	No
3	51.2GB/sec	256 Bit	800 MHz	GDDR3			Radeon HD 2900 Pro	No
4	36.8GB/sec	128 Bit	1150 MHz	GDDR4	Radeon HD 2600	XT Diamond Edition		No
5	22.4GB/sec	128 Bit	700 MHz	GDDR3	Radeon HD 2600	XT		No
6	35.2GB/sec	128 Bit	1100 MHz	GDDR4	Radeon HD 2600	XT 256MB GDDR4		No
	Open_GL	PSU	Pixel_Rate	Power_Connector	Process	ROPs	Release_Date	Release_Price
1	3.3	450 Watt & 38 Amps	12 GPixel/s	None	55nm	16	\n01-Mar-2009	
2	3.1	550 Watt & 35 Amps	12 GPixel/s	None	80nm	16	\n14-May-2007	
3	3.1	550 Watt & 35 Amps	10 GPixel/s	None	80nm	16	\n07-Dec-2007	
4	3.3		3 GPixel/s	None	65nm	4	\n01-Jul-2007	
5	3.1	400 Watt & 25 Amps	3 GPixel/s	None	65nm	4	\n28-Jun-2007	
6	3.3	400 Watt & 26 Amps	3 GPixel/s	None	65nm	4	\n26-Jun-2007	
	Resolution_wxH	SLI_Crossfire	Shader	TMUs	Texture_Rate	VGA_Connection		
1	2560x1600	Yes	4	64	47 GTexel/s	0		
2	2560x1600	Yes	4	16	12 GTexel/s	0		
3	2560x1600	Yes	4	16	10 GTexel/s	0		
4	2560x1600	Yes	4	8	7 GTexel/s	0		
5	2560x1600	Yes	4	8	6 GTexel/s	0		
6	2560x1600	Yes	4	8	6 GTexel/s	0		

> |

Hình 3.1. Một vài dòng đầu tiên của dữ liệu

3.2 Chọn lọc dữ liệu

Chọn ra 10 thuộc tính cần thiết từ 34 thuộc tính trong dữ liệu GPUs ban đầu: **Name**, **Best_Resolution**, **Core_Speed**, **Max_Power**, **Memory**, **Memory_Bandwidth**, **Memory_Speed**, **Manufacturer**, **Release_Date**, **Release_Price**.

```
1 dat<-dat[,c("Name", "Best_Resolution", "Core_Speed", "Max_Power", "Memory", "
Memory_Bandwidth", "Memory_Speed", "Manufacturer", "Release_Date", "Release_Price
")]
2 View(dat)
```

Name	Best_Resolution	Core_Speed	Max_Power	Memory	Memory_Bandwidth	Memory_Speed	Manufacturer	Release_Date	Release_Price
1 GeForce GTS 150		738 MHz	141 Watts	1024 MB	64GB/sec	1000 MHz	Nvidia	01-Mar-2009	
2 Radeon HD 2900 XT 512MB	1366 x 768	-	215 Watts	512 MB	106GB/sec	828 MHz	AMD	14-May-2007	
3 Radeon HD 2900 Pro	1366 x 768	-	200 Watts	512 MB	51.2GB/sec	800 MHz	AMD	07-Dec-2007	
4 Radeon HD 2600 XT Diamond Edition	1024 x 768	-		256 MB	36.8GB/sec	1150 MHz	AMD	01-Jul-2007	
5 Radeon HD 2600 XT	1024 x 768	-	45 Watts	256 MB	22.4GB/sec	700 MHz	AMD	28-Jun-2007	
6 Radeon HD 2600 XT 256MB GDDR4	1024 x 768	-	50 Watts	256 MB	35.2GB/sec	1100 MHz	AMD	26-Jun-2007	
7 Radeon HD 4890 Sapphire Vapor-X OC 2GB Edition	1920 x 1080	870 MHz	190 Watts	2048 MB	134.4GB/sec	1050 MHz	AMD	13-Jul-2009	
8 Radeon HD 2900 GT	1024 x 768	-	150 Watts	256 MB	51.2GB/sec	800 MHz	AMD	06-Nov-2007	
9 FirePro D300	1920 x 1080	-	150 Watts	2048 MB	160GB/sec	1250 MHz	AMD	18-Jan-2014	
10 Radeon 7000 64mb		-	32 Watts	64 MB	2.9GB/sec	366 MHz	AMD	02-Jan-2001	
11 Quadro4 980 XGL		-		128 MB	5.2GB/sec	325 MHz	Nvidia	01-Nov-2002	
12 Tesla M2090	1920 x 1080	630 MHz	250 Watts	6144 MB	177.6GB/sec	925 MHz	Nvidia	25-Jul-2011	
13 Tesla K20		705 MHz	225 Watts	5120 MB	168GB/sec	1050 MHz	Nvidia	01-Nov-2012	
14 Tesla K40c	2560 x 1900	706 MHz	245 Watts	12288 MB	288.4GB/sec	1502 MHz	Nvidia	12-Nov-2013	
15 All-in-Wonder Radeon 7500		-		64 MB	5.8GB/sec	360 MHz	AMD	22-Jan-2002	
16 Radeon R7 250v2 MSI OC 2GB + Radeon R7 7870K Dual	1600 x 900	1050 MHz	150 Watts	3072 MB	57.6GB/sec	900 MHz	AMD	28-May-2015	
17 Tesla K10		-	300 Watts	8192 MB	320GB/sec	1250 MHz	Nvidia	15-May-2012	
21 Tesla K20X		732 MHz	235 Watts	6144 MB	249.6GB/sec	1300 MHz	Nvidia	12-Nov-2012	
19 Irs i3 5157U	1366 x 768	300 MHz	28 Watts		34.1GB/sec	1067 MHz	Intel	01-Sep-2015	
20 Radeon 3800 XT	1024 x 768	-	60 Watts	256 MB	23.4GB/sec	730 MHz	AMD	01-Oct-2003	
21 Tesla C2070		575 MHz	235 Watts	6144 MB	144GB/sec	750 MHz	Nvidia	01-Sep-2010	
22 Tesla C2075		575 MHz	247 Watts	6144 MB	144GB/sec	750 MHz	Nvidia	01-Jul-2011	
23 Quadro4 750 XGL		-		128 MB	3.6GB/sec	225 MHz	Nvidia	19-Feb-2002	
24 Quadro4 900 XGL		-		128 MB	5.2GB/sec	325 MHz	Nvidia	01-Nov-2002	
25 Irs i5 5350U	1366 x 768	300 MHz	15 Watts		34.1GB/sec	1067 MHz	Intel	01-Sep-2015	
26 Quadro4 780 XGL		-		128 MB	4.4GB/sec	275 MHz	Nvidia	01-Nov-2002	

Hình 3.2. Vài phần tử của dữ liệu sau khi chọn lọc

3.3 Xử lý định dạng dữ liệu

Sử dụng thư viện janitor và tidyr của R để thực hiện định dạng lại tên biến, xóa các hàng và cột trống (nếu có), tính toán sơ bộ.

```
1 library(tidyr)
2 library(janitor)
3 #định dạng lại tên biến
4 dat2 <- clean_names(dat)
5 head(dat2)
6 #xóa hàng và cột trống
7 dat3 <- remove_empty(dat2, which = c("rows","cols"),quiet=FALSE)
8 head(dat3)
```

```
> head(dat3)
  name best_resolution core_speed max_power memory memory_bandwidth memory_speed manufacturer release_date release_price
1   GeForce GTS 150      738 MHz 141 Watts 1024 MB      64GB/sec      1000 MHz      Nvidia  \n01-Mar-2009
2   Radeon HD 2900 XT 512MB 1366 x 768  \n-  215 Watts  512 MB      106GB/sec      828 MHz      AMD  \n14-May-2007
3   Radeon HD 2900 Pro 1366 x 768  \n-  200 Watts  512 MB      51.2GB/sec      800 MHz      AMD  \n07-Dec-2007
4 Radeon HD 2600 XT Diamond Edition 1024 x 768  \n-  256 MB      36.8GB/sec      1150 MHz      AMD  \n01-Jul-2007
5   Radeon HD 2600 XT 1024 x 768  \n-  45 Watts  256 MB      22.4GB/sec      700 MHz      AMD  \n28-Jun-2007
6   Radeon HD 2600 XT 256MB GDDR4 1024 x 768  \n-  50 Watts  256 MB      35.2GB/sec      1100 MHz      AMD  \n26-Jun-2007
```

Hình 3.3. Vài phần tử đầu của dữ liệu sau khi định dạng

Mẫu `best_resolution` đề cập đến độ phân giải tối ưu mà GPU hoặc thiết bị có thể hỗ trợ, bao gồm số lượng pixel trên mỗi chiều (chiều rộng x chiều cao). Do đó ta đổi tên mẫu này thành `number_of_pixels` và tính toán số điểm ảnh (pixels).

```
1 # Đổi tên best_resolution thành number_of_pixels
2 names(dat3)[ names(dat3) == "best_resolution"] <- "number_of_pixels"
3 # Thực hiện phép nhân số pixel hai chiều ngang và dọc
```

```
4 dat3$number_of_pixels <- sapply (strsplit(dat3$number_of_pixels,"x"), function(x) as.numeric (x [1])*as.numeric (x[2]))
```

Đối với các mẫu `core_speed`, `max_power`, `memory`, `memory_bandwidth`, `memory_speed` để dễ dàng thao tác tính toán, cần tách mỗi mẫu thành hai cột gồm phần **giá trị** và phần **đơn vị**:

```
1 #loai bo gia tri khuyet bieu dieu boi "-"
2 dat3$core_speed[grepl("-", dat3$core_speed)] <- ""
3 #chia thanh 2 cot core_speed_value va core_speed_unit
4 dat3 <- separate(dat3,col = core_speed, into = c("core_speed_value",
5         "core_speed_unit"),sep=" ", fill="right")
6 #doi gia tri thanh dang so
7 dat3$core_speed_value <- as.numeric(dat3$core_speed_value)
8 #thuc hien tuong tu cho cac cot con lai
9 dat3 <- separate(dat3, col = max_power, into = c("max_power_value",
10         "max_power_unit"), sep=" ", fill = "right")
11 dat3$max_power_value <- as.numeric(dat3$max_power_value)
12
13 dat3 <- separate(dat3, col = memory, into = c("memory_value",
14         "memory_unit"), sep=" ", fill = "right", extra = "drop")
15 dat3$memory_value <- as.numeric(dat3$memory_value)
16
17 dat3 <- separate(dat3, col = memory_bandwidth,
18         into = c("memory_bandwidth_value", "memory_bandwidth_unit"),
19         sep = "(?<=\\d)(?=[A-Za-z])", fill = "right")
20 dat3$memory_bandwidth_value <- as.numeric(dat3$memory_bandwidth_value)
21
22 dat3 <- separate(dat3, col = memory_speed ,
23         into = c("memory_speed_value", "memory_speed_unit"),
24         sep = " ", fill = "right")
25 dat3$memory_speed_value <- as.numeric (dat3$memory_speed_value)
```

Trong đó:

- function `separate()` của thư viện `tidyr`
- `separate()` chứa `col` là tên cột cần phân tách, `into` là vector chứa tên của cột mới, `sep` kí tự dùng để phân tách cột dữ liệu, `fill="right"` điền giá trị mới vào bên phải của kết quả

name	number_of_plats	core_speed_value	core_speed_unit	max_power_value	max_power_unit	memory_value	memory_unit	memory_bandwidth_value	memory_bandwidth_unit
1 GeForce GTS 150	NA	738	MHz	141	Watts	1024	MB	64.0	GB/sec
2 Radeon HD 2900 XT 512MB	1049088	NA	NA	215	Watts	512	MB	106.0	GB/sec
3 Radeon HD 2900 Pro	1049088	NA	NA	200	Watts	512	MB	51.2	GB/sec
4 Radeon HD 2600 XT Diamond Edition	786432	NA	NA	NA	NA	256	MB	16.0	GB/sec
5 Radeon HD 2600 XT	786432	NA	NA	45	Watts	256	MB	22.4	GB/sec
6 Radeon HD 2600 XT 256MB GDDR4	786432	NA	NA	50	Watts	256	MB	35.2	GB/sec
7 Radeon HD 4890 Sapphire Vapor-X OC 2GB Edition	2073600	870	MHz	190	Watts	2048	MB	134.4	GB/sec
8 Radeon HD 2900 GT	786432	NA	NA	150	Watts	256	MB	51.2	GB/sec
9 FirePro 2270	2073600	NA	NA	150	Watts	3088	MB	160.0	GB/sec
10 Radeon 7000 64mb	NA	NA	NA	32	Watts	64	MB	2.0	GB/sec
11 Quadrol 980 NGL	NA	NA	NA	NA	NA	128	MB	5.2	GB/sec
12 Tesla M2090	2073600	650	MHz	250	Watts	6144	MB	177.6	GB/sec
13 Tesla K20	NA	705	MHz	225	Watts	5120	MB	168.0	GB/sec
14 Tesla K80c	4096000	706	MHz	245	Watts	13288	MB	288.8	GB/sec
15 All-in-Wonder Radeon 7500	NA	NA	NA	NA	NA	64	MB	5.8	GB/sec
16 Radeon R7 250 v2 MSR-OC 2GB + Radeon R7 7870K Dual	1440000	1050	MHz	150	Watts	3072	MB	57.6	GB/sec
17 Tesla K10	NA	NA	NA	300	Watts	8192	MB	320.0	GB/sec
18 Tesla K20K	NA	732	MHz	235	Watts	6144	MB	249.6	GB/sec
19 Ilex D 616TU	1049088	300	MHz	28	Watts	NA	NA	14.1	GB/sec
20 Radeon 9800 XT	786432	NA	NA	60	Watts	256	MB	23.4	GB/sec
21 Tesla C2070	NA	575	MHz	238	Watts	6144	MB	144.0	GB/sec
22 Tesla C2075	NA	575	MHz	247	Watts	6144	MB	144.0	GB/sec
23 Quadrol 795 NGL	NA	NA	NA	NA	NA	128	MB	3.6	GB/sec
24 Quadrol 900 NGL	NA	NA	NA	NA	NA	128	MB	5.2	GB/sec
25 Ilex D 630DU	1049088	300	MHz	15	Watts	NA	NA	14.1	GB/sec

Hình 3.4. Kết quả sau khi tách cột

Kiểm tra số lượng đơn vị của một thuộc tính bằng lệnh `table()`:

```
1 table(dat3$core_speed_unit)
2 table(dat3$max_power_unit)
3 table(dat3$memory_unit)
4 table(dat3$memory_bandwidth_unit)
5 table(dat3$memory_speed_unit)
```

```
> table(dat3$core_speed_unit)
MHz
2470
> table(dat3$max_power_unit)
watts
2781
> table(dat3$memory_unit)
MB
2986
> table(dat3$memory_bandwidth_unit)
GB/sec MB/sec
3281      4
> table(dat3$memory_speed_unit)
MHz
3301
>
```

Hình 3.5. Số lượng các đơn vị

Trong những thuộc tính trên, chỉ có `memory_bandwidth_unit` là chứa hai đơn vị $[MB/sec]$ và $[GB/sec]$. Do đó ta cần đồng nhất dữ liệu bằng cách đưa thuộc tính này về cùng một đơn vị là $[GB/sec]$ bằng hàm `ifelse(test,yes,no)` trong R.

```
1 dat3$memory_bandwidth_value <- ifelse(dat3$memory_bandwidth_unit == "MB/sec", dat3$memory_bandwidth_value/1024, dat3$memory_bandwidth_value)
2 dat3$memory_bandwidth_unit <- ifelse(dat3$memory_bandwidth_unit == "MB/sec", "GB/sec", dat3$memory_bandwidth_unit)
```

```
> dat3$memory_bandwidth_value <- ifelse(dat3$memory_bandwidth_unit == "MB/sec", dat3$memory_bandwidth_value/1024, dat3$memory_bandwidth_value)
> dat3$memory_bandwidth_unit <- ifelse(dat3$memory_bandwidth_unit == "MB/sec", "GB/sec", dat3$memory_bandwidth_unit)
> table(dat3$memory_bandwidth_unit)

GB/sec
3285
> |
```

Hình 3.6. Kết quả sau khi đồng nhất đơn vị

Cột dữ liệu **release_date** chuyển từ định dạng chuỗi về định dạng ngày tháng bằng lệnh **as.Date()**. Sau đó chuyển thành kiểu dữ liệu số và tính toán đưa về đơn vị [năm]

```
1 #tu dinh dang chuoi -> dinh dang ngay thang
2 dat3$release_date <- as.Date(sub("\\s*\\n", "", dat3$release_date),
3                               format = "%d-%b-%Y")
4 #tinh gia tri cua cot theo don vi "nam"
5 dat3$release_date <- as.numeric(format(dat3$release_date, "%Y")) +
6                               as.numeric(format(dat3$release_date, "%m")) / 12
```

Với cột dữ liệu **release_price**, xóa bỏ ký tự \$ và chuyển thành định dạng số để dễ dàng thực hiện tính toán

```
1 dat3$release_price <- gsub("\\$", "", dat3$release_price)
2 dat3$release_price <- as.numeric(dat3$release_price)
```

Cuối cùng, kiểm tra lại định dạng của dữ liệu bằng lệnh **str()**

```
Console Terminal Background Jobs
R 4.3.3 - /
> str(dat3)
'data.frame': 3406 obs. of 15 variables:
 $ name : chr "GeForce GTS 150" "Radeon HD 2900 XT 512MB" "Radeon HD 2900 Pro" "Radeon HD 2600 XT Diamond Edition" ...
 $ number_of_pixels : num NA 1049088 1049088 786432 786432 ...
 $ core_speed_value : chr "738" "" "" "" ...
 $ core_speed_unit : chr "MHz" "NA" "NA" "NA" ...
 $ max_power_value : num 141 215 200 NA 45 50 190 150 150 32 ...
 $ max_power_unit : chr "watts" "watts" "watts" "NA" ...
 $ memory_value : num 1024 512 512 256 256 ...
 $ memory_unit : chr "MB" "MB" "MB" "MB" ...
 $ memory_bandwidth_value: num 64 106 51.2 36.8 22.4 ...
 $ memory_bandwidth_unit: chr "GB/sec" "GB/sec" "GB/sec" "GB/sec" ...
 $ memory_speed_value : chr "1000" "828" "800" "1150" ...
 $ memory_speed_unit : chr "MHz" "MHz" "MHz" "MHz" ...
 $ manufacturer : chr "Nvidia" "AMD" "AMD" "AMD" ...
 $ release_date : num 2009 2007 2008 2008 2008 ...
 $ release_price : num NA NA NA NA NA NA NA NA ...
```

Hình 3.7. Kết quả kiểm tra dữ liệu

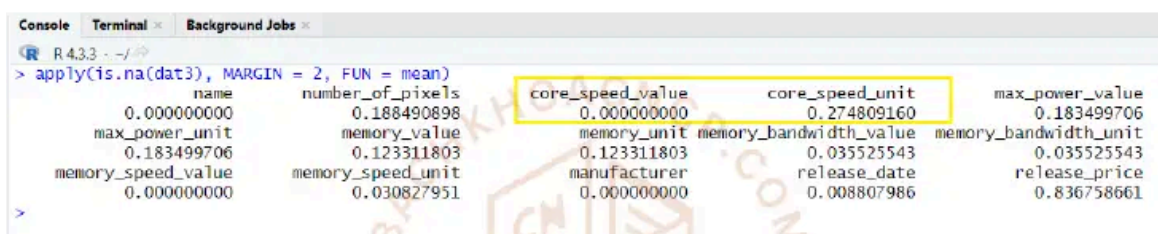
3.4 Xử lý dữ liệu khuyết

Trong R, các giá trị "N/A" thường xuất hiện khi có dữ liệu bị thiếu trong data frame. Việc này xảy ra khi dữ liệu đó không có sẵn hoặc bị thiếu. Có vài cách để xử lý dữ liệu khuyết:

- Loại bỏ dữ liệu khuyết
- Thay thế dữ liệu khuyết bằng giá trị trung bình hoặc trung vị của các phần tử không khuyết
- Thay thế bằng một giá trị cố định khác

Đầu tiên, ta cần kiểm tra tỉ lệ dữ liệu bị khuyết của các thuộc tính bằng lệnh `apply()`. Bởi vì ban đầu ở mục 3.3 ta đã loại bỏ những giá trị khuyết "-" của `core_speed_value` nên tỉ lệ khuyết của thuộc tính này bằng với tỉ lệ khuyết của `core_speed_unit`.

```
1 apply(is.na(dat3), MARGIN = 2, FUN = mean)
```



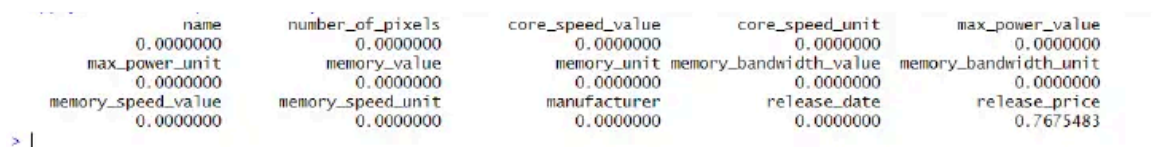
name	number_of_pixels	core_speed_value	core_speed_unit	max_power_value	max_power_unit	memory_value	memory_unit	memory_bandwidth_value	memory_bandwidth_unit
0.00000000	0.188490898	0.00000000	0.274809160	0.183499706	0.183499706	0.123311803	0.123311803	0.035525543	0.035525543

Hình 3.8. Tỉ lệ bị khuyết của dữ liệu

Sau đó, ta tiến hành xử lý dữ liệu khuyết bằng cách loại bỏ dữ liệu thông qua hàm `na.omit()`, riêng thuộc tính `release_price` không loại bỏ khuyết vì tỉ lệ khuyết quá lớn.

```
1 # Tạo Data frame "temp" từ dat3 trừ release_price
2 temp <- dat3[, !(names(dat3) %in% "release_price")]
3 #loại bỏ dữ liệu khuyết
4 temp <- na.omit(temp)
5 #gộp 2 data frame lại
6 dat3 <- dat3[rownames(temp), ]
```

Sau khi xử lý các giá trị khuyết, kiểm tra lại dữ liệu bằng hàm `apply()`



name	number_of_pixels	core_speed_value	core_speed_unit	max_power_value	max_power_unit	memory_value	memory_unit	memory_bandwidth_value	memory_bandwidth_unit
0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000

Hình 3.9. Kết quả kiểm tra khuyết

4 Thống kê mô tả

4.1 Các giá trị đặc trưng của mẫu

Ta dùng các hàm `sapply()`, `mean()`, `var()` và `fivenum()` để tính(hầu hết) các giá trị đặc trưng của mẫu định lượng từ file số liệu đã xử lý:

```
1 data <- read_csv("my_data.csv");
2 qtt_data <- data[, c("number_of_pixels", "core_speed_value", "max_power_value", "
  memory_value", "memory_bandwidth_value", "memory_speed_value", "release_date")]
3
4 # Tìm giá trị trung bình
5 avg = sapply(qtt_data, mean)
6
7 # Tìm phương sai
8 s2 = sapply(qtt_data, var)
9
10 # Tìm 5 thông số lần lượt, bao gồm
11 # Min, Tứ phân vị 0.25, Trung vị, Tứ phân vị 0.75, Max
12 fivenum = sapply(qtt_data, fivenum)
13
14 # Tạo bảng chứa các giá trị đặc trưng
15 # và làm tròn đến 2 chữ số thập phân
16 fivenum = t(fivenum)
17 t = split(fivenum, rep(1:ncol(fivenum), each = nrow(fivenum)))
18 names(t) = c("Min", "Quar1", "Quar2", "Quar3", "max")
19 table = round(data.frame(avg, s2, t), 2)
20 show(table)
```

Description: df [7 x 7]

	Avg	s2	Min	Quar1	Quar2	Quar3	Max
number_of_pixels	2313405.78	2.580255e+12	480000.00	1440000.00	2073600.00	2073600.00	9210000.00
core_speed_value	980.88	6.413929e+04	200.00	800.00	1000.00	1106.00	1784.00
max_power_value	144.88	1.086828e+04	1.00	65.00	130.00	190.00	780.00
memory_value	3308.73	7.700892e+08	128.00	1280.00	2048.00	4096.00	24576.00
memory_bandwidth_value	166.13	1.922599e+04	4.00	72.00	128.30	224.40	1280.00
memory_speed_value	1307.28	1.466370e+05	400.00	1000.00	1350.00	1650.00	2127.00
release_date	2013.68	4.690000e+00	2000.92	2012.25	2013.83	2015.42	2017.33

7 rows

Hình 4.1. Các giá trị đặc trưng thu được

Riêng thông số `Release_Price` do có nhiều giá trị khuyết nên ta sẽ sử dụng hàm `summary()` để xác định:

```
1 summary(data$release_price)
```

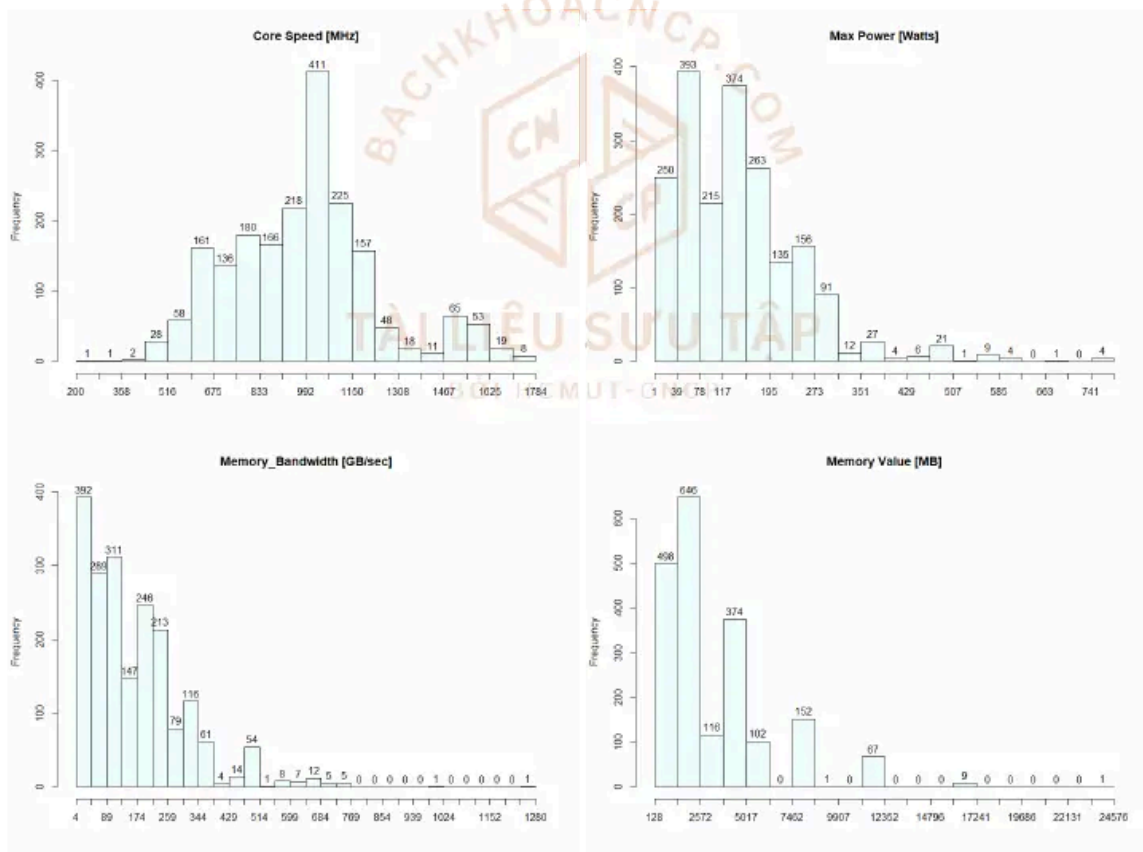
```
Min. 1st Qu. Median Mean 3rd Qu. Max. NA's
23.0 150.0 249.0 385.8 449.0 14999.0 1509
```

Hình 4.2. Các giá trị đặc trưng của `Release_Price`

4.2 Các loại đồ thị

Dùng hàm **hist()**, ta có thể quan sát được một cách trực quan sự phân phối của một số mẫu định lượng và đưa ra nhận xét:

```
1 # Hàm tự định nghĩa để vẽ đồ thị Histogram
2 plotHist <- function(sample, name, num_of_breaks){
3   sample_x_axis = floor(seq(floor(min(sample)),
4     max(sample), length.out = num_of_breaks+1))
5   hist(sample, sample_x_axis, freq = TRUE, xlab = name,
6     labels = TRUE, xaxt="n", col = "azure")
7   axis(1, at = sample_x_axis)
8 }
9
10 plotHist(data$core_speed_value, "Core Speed [MHz]", 20)
11 plotHist(data$max_power_value, "Max Power [Watts]", 20)
12 plotHist(data$memory_bandwidth_value, "Memory_Bandwidth [GB/sec]", 30)
13 plotHist(data$memory_value, "Memory Value [MB]", 20)
```



Hình 4.3. Biểu đồ phân phối tần số một số thông số của mẫu

Nhận xét: Hầu hết các thông số có phân phối bị lệch trái như **max_power_value**, **memory_power_value** và **memory_value** với phần lớn các giá trị nằm trong khoảng < 300 Watts đối với **max_power_value**, < 400 GB/sec đối với **memory_power_value** và < 4096 MB đối

với **memory_value**, riêng **core_speed_value** lại có phân phối tương đối chuẩn tập trung trong khoảng từ 900 MHz đến 1000 MHz.

4.3 Mối liên hệ giữa các biến

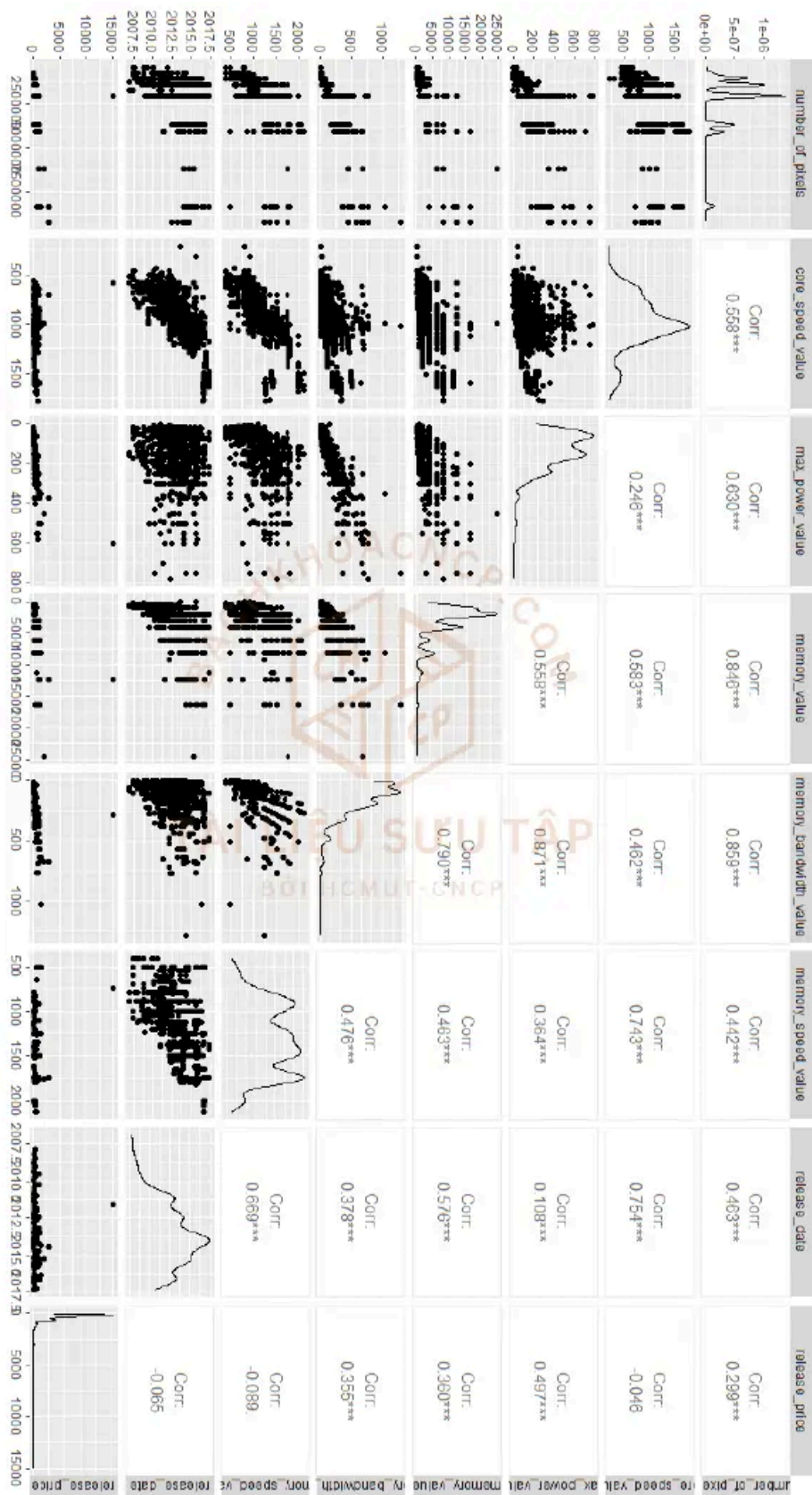
Ta sử dụng hàm **ggpairs()** trong thư viện **GGally** để tạo ma trận biểu đồ tán xạ tổng hợp thể hiện mối liên hệ giữa các thông số (trang tiếp theo):

```
1 library(GGally)
2 qtt_data <- data[, c("number_of_pixels", "core_speed_value", "max_power_value", "
  memory_value", "memory_bandwidth_value", "memory_speed_value", "release_date", "
  release_price")]
3 ggpairs(qtt_data)
```

Giới thiệu về **ggpairs**: Hàm **ggpairs()** mặc định sẽ trả về một ma trận gồm ba thành phần chính là bảng giá trị phía trên đường chéo chính mang hệ số tương quan của từng cặp thông số, đồ thị nằm trên đường chéo chính là biểu đồ đường của từng hàm số và nằm dưới đường chéo chính là biểu đồ tán xạ (scatter plot) của từng cặp thông số.

Nhận xét:

- Ba cặp thông số có tương quan mạnh với nhau là ($r > 0.8$):
 - **memory_bandwidth_value** và **number_of_pixels**:
Có thể khả năng xử lý nhiều điểm ảnh cao đồng nghĩa cần băng thông lớn để đáp ứng lượng dữ liệu.
 - **memory_value** và **number_of_pixels**:
Có thể khả năng xử lý nhiều điểm ảnh cao đồng nghĩa cần băng thông lớn để đáp ứng lượng dữ liệu.
 - **memory_bandwidth_value** và **max_power_value**:
Có thể là do bộ nhớ có băng thông cao sẽ yêu cầu nhiều điện năng hơn.
- Năm cặp thông số có tương quan khá mạnh với nhau là ($0.8 > r > 0.6$):
 - **max_power_value** và **number_of_pixels**:
Có thể để xử lý nhiều điểm ảnh thì cần cung cấp nhiều điện năng hơn
 - **memory_value** và **memory_bandwidth_value**
 - **memory_speed_value** và **core_speed_value**
 - **core_speed_value** và **release_date**:
 - **memory_speed_value** và **release_date**:
Có thể là do GPU ra đời gần đây sẽ có tốc độ nhanh hơn
- Thông số **release_price** ít tương quan với những thông số còn lại:
Có thể là do giá của một GPU phụ thuộc vào nhiều thông số song ta chỉ so nó với từng thông số một nên dẫn đến sự tương quan của nó với từng thông số là nhỏ.



Hình 4.4. Mối liên hệ giữa các thông số và giá trị của các thông số

5 Thống kê suy diễn

5.1 Tìm khoảng tin cậy một mẫu

5.1.1 Mục tiêu

- Nắm được các khái niệm liên quan đến lý thuyết ước lượng và bài toán tìm khoảng ước lượng tin cậy của một tỉ lệ trong tổng thể.
- Biết phương pháp giải quyết bài toán tìm khoảng tin cậy, cụ thể cho một mẫu.
- Nắm được căn bản cách ứng dụng ngôn ngữ R vào bài toán.

5.1.2 Bài toán

Xét mẫu **Memory_Value** từ tập dữ liệu đã được tiền xử lý. Tìm khoảng ước lượng hai phía cho tỉ lệ những GPU có giá trị **Memory_Value** từ 4096 MB trở lên trong tổng thể với độ tin cậy 95%.

5.1.3 Các lệnh R sử dụng

Các lệnh nhóm sử dụng để giải quyết bài toán được liệt kê trong bảng sau:

Thư viện và lệnh R	Chức năng
BSDA	Thư viện cung cấp lệnh <code>z.test()</code>
<code>prop.test()</code>	Kiểm định cho một tỉ lệ bao gồm khoảng ước lượng
<code>mean()</code>	Tìm trung bình của mẫu
<code>sd()</code>	Tìm độ lệch chuẩn của mẫu
<code>qnorm()</code>	Tìm giá trị z-score ứng với độ tin cậy
<code>cat</code>	Định dạng lại kết quả đầu ra

Bảng 5.1. Các lệnh R sử dụng trong bài toán

5.1.4 Tiền xử lý bài toán

Vì tỉ lệ các GPU của mẫu có giá trị **Memory_Value** trên 4096 MB cũng là biến ngẫu nhiên tuân theo phân phối nhị thức, mà kích thước mẫu lớn nên sử dụng định lý giới hạn trung tâm, biến ngẫu nhiên này có thể được xấp xỉ về phân phối chuẩn.

Đây là bài toán tìm khoảng tin cậy hai phía, sử dụng cơ sở lý thuyết, nhóm đưa ra lời giải trên R. Ngoài ra, nhóm sẽ sử dụng hàm được cung cấp sẵn bởi R là `prop.test()` để so sánh kết quả.

5.1.5 Giải bài toán trên R

Trước hết, ta tính toán các giá trị cần thiết như sai số chuẩn, giá trị `z_score` với độ tin cậy tương ứng.

```
1 # Đọc dữ liệu đã được xử lý và lấy mẫu của giá trị memory_value
2 df <- read.csv("my_data.csv")
3 mem <- df$memory_value
4
5 # Lấy ra các mẫu có giá trị memory_value >= 4096 MB
6 mem_larger <- mem[mem >= 4096]
```



```
7
8 # Tính kích thước mẫu và tỉ lệ các GPU có memory_value từ 4096 MB trở lên
9 size_total <- length(mem)
10 size_larger <- length(mem_larger)
11
12 f <- size_larger / size_total
13
14 # Tính giá trị z_score ứng với alpha/2
15 confidence <- 0.95
16 alpha <- 1 - confidence
17
18 z_score <- qnorm(1 - alpha / 2)
```

Trình bày kết quả tính toán được

```
1 # Tính khoảng tin cậy và trình bày kết quả
2 SE <- sqrt(f * (1 - f) / size_total)
3 epsilon <- z_score * SE
4
5 lower_bound <- f - epsilon
6 upper_bound <- f + epsilon
7 cat("The 95% confidence interval of p is (",
8     lower_bound, ", ", upper_bound, ")", sep = "")
```

Kết quả mà nhóm thu được với khoảng tin cậy cho tỉ lệ tổng thể các GPU có giá trị **Memory_Value** từ 4096 GB trở lên là:

```
1 The 95% confidence interval of p is (0.3378987, 0.3803109)
```

Sử dụng thêm hàm `prop.test()` được cung cấp sẵn bởi R, nhóm so sánh với khoảng tin cậy tỉ lệ vừa tìm được.

```
1 prop.test(
2   x = size_larger,
3   n = size_total,
4   alternative = "two.sided",
5   conf.level = confidence
6 )
```

Kết quả thu được:

```
1 1-sample proportions test with continuity correction
2
3 data:  size_larger out of size_total, null probability 0.5
4 X-squared = 155.55, df = 1, p-value < 2.2e-16
5 alternative hypothesis: true p is not equal to 0.5
6 95 percent confidence interval:
7  0.3379418 0.3808238
8 sample estimates:
9      p
10 0.3591048
```

5.1.6 Nhận xét

Dữ liệu từ dòng 95 percent confidence interval cho thấy khoảng tin cậy (0.3379418;0.3808238) tương đương với kết quả đã tìm được bằng cơ sở lý thuyết, với sai lệch rất nhỏ.

Kết quả sự sai khác nhỏ này là do ngay từ ban đầu, ta đang sử dụng định lý giới hạn trung tâm để xấp xỉ cho phân phối nhị thức về phân phối chuẩn.

5.1.7 Kết luận

Đối với bài toán này, nhóm đã được thực hành các lệnh với ngôn ngữ thống kê R, tìm ra được khoảng ước lượng cho trung bình tổng thể, áp dụng từ cơ sở lý thuyết về khoảng tin cậy đã được học.

5.2 Kiểm định hai mẫu

5.2.1 Mục tiêu

- Nắm và hiểu được khái niệm liên quan đến lý thuyết kiểm định giả thuyết thống kê.
- Biết phương pháp giải quyết bài toán kiểm định trung bình, tỉ lệ một mẫu, hai mẫu.
- Nắm được căn bản cách ứng dụng ngôn ngữ R vào bài toán

5.2.2 Bài toán

Xét mẫu dựa trên giá trị **Memory_Banwidth_Value**, phân loại ra hai mẫu con dựa theo **Manufacturer** (nhà sản xuất GPU) theo thứ tự là **Nvidia** và **AMD**. Tính toán trên mẫu cho thấy giá trị trung bình về giá trị Memory Bandwidth của mẫu thứ nhất lớn hơn của mẫu thứ hai. Với mức ý nghĩa $\alpha = 5\%$, có thể coi giá trị trung bình về Memory Bandwidth của tổng thể các GPU do hãng sản xuất Nvidia lớn hơn hãng AMD không.

5.2.3 Các lệnh R sử dụng

Các lệnh nhóm sử dụng để giải quyết bài toán được liệt kê trong bảng sau:

Thư viện và lệnh R	Chức năng
BSDA	Thư viện cung cấp lệnh <code>z.test()</code>
<code>z.test()</code>	Tìm khoảng tin cậy của trung bình tổng thể
<code>mean()</code>	Tìm trung bình của mẫu
<code>sd()</code>	Tìm độ lệch chuẩn của mẫu
<code>qnorm()</code>	Tìm giá trị z-score ứng với độ tin cậy
<code>length()</code>	Tìm kích thước vector mẫu
<code>print()</code> và <code>paste()</code>	Hiển thị kết quả ra console

Bảng 5.2. Các lệnh R sử dụng trong bài toán

5.2.4 Tiền xử lý bài toán

Kiểm tra phân phối của mẫu

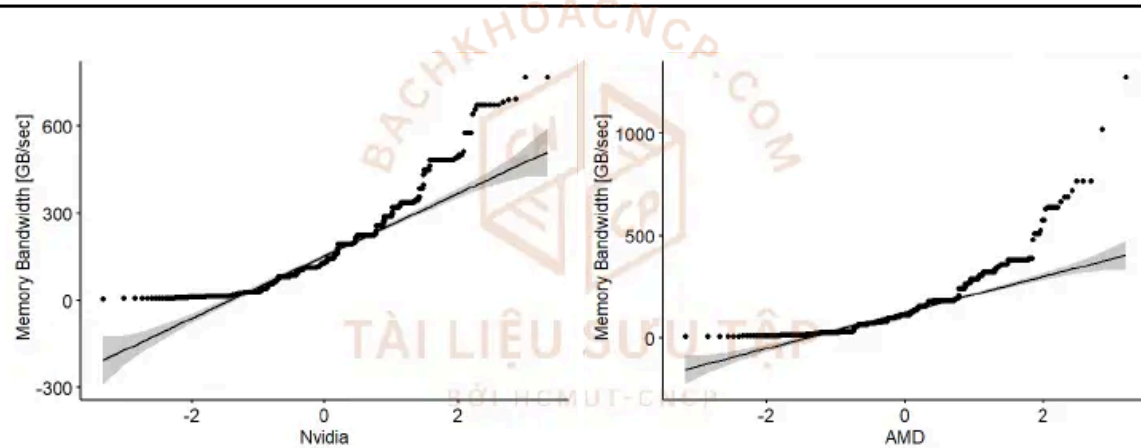
Trước hết, nhóm kiểm tra các giá trị **Memory_Bandwidth_Value** của GPU sản xuất từ hãng Nvidia và AMD có tuân theo phân phối chuẩn hay không.

Có nhiều phương pháp để kiểm tra phân phối chuẩn, nhóm sẽ sử dụng hai phương pháp:

- Phương pháp đồ thị: Sử dụng Biểu đồ Q-Q (Quantile-Quantile)
- Phương pháp thống kê: Sử dụng Kiểm định Shapiro-Wilk

Sử dụng Biểu đồ Q-Q

```
1 # Trích dữ liệu memory_bandwidth_value theo hãng Nvidia và AMD
2 df <- read.csv("my_data.csv")
3 nvidia <- df$memory_bandwidth_value[df$manufacturer == "Nvidia"]
4 amd <- df$memory_bandwidth_value[df$manufacturer == "AMD"]
5
6 library(ggpubr)
7 ggqqplot(nvidia, ylab = "Memory Bandwidth [GB/sec]")
8 ggqqplot(amd, ylab = "Memory Bandwidth [GB/sec]")
```



Hình 5.1. Biểu đồ QQ-plot cho mẫu của Nvidia và AMD

Ta thấy các giá trị tứ phân vị nằm khá xa lệch đường thẳng kì vọng của phân phối chuẩn nên những giá trị **Memory_Bandwidth** của GPU từ hai hãng Nvidia và AMD có khả năng cao không theo phân phối chuẩn.

Để kết luận chắc chắn hơn, nhóm sử dụng thêm kiểm định Shapiro-Wilk.

Kiểm định Shapiro-Wilk

Giả thuyết kiểm định:

- Giả thuyết H_0 : Biến kiểm định tuân theo phân phối chuẩn
- Giả thuyết H_1 : Biến kiểm định không tuân theo phân phối chuẩn

```
1 shapiro.test(nvidia)
2 shapiro.test(amd)
```

```
1 Shapiro-Wilk normality test
2
3 data:  nvidia
4 W = 0.88913, p-value < 2.2e-16
5
6 Shapiro-Wilk normality test
7
8 data:  amd
9 W = 0.80454, p-value < 2.2e-16
```

Kết quả cho thấy giá trị $p_value = 2.2e - 16 < 0.05$. Do đó có thể bác bỏ giả thuyết H_0 và chấp nhận giả thuyết H_1 rằng các biến ngẫu nhiên đang khảo sát đến từ hãng Nvidia và AMD không tuân theo phân phối chuẩn.

Chọn phương án giải quyết

Bài toán kiểm định trung bình hai mẫu trên có những đặc điểm sau:

- Hai mẫu độc lập, do mỗi GPU chỉ đến từ một nhãn hàng)
- Giá trị **Memory_Bandwidth_Value** có phân phối tùy ý khác phân phối chuẩn.
- Kích thước hai mẫu lớn.
- Chưa biết phương sai của hai tổng thể.

Do đó, áp dụng kết quả của định lý giới hạn trung bình, chênh lệch giữa trung bình hai tổng thể xấp xỉ phân phối chuẩn, nhóm sẽ sử dụng tiêu chí kiểm định **z.test** cho bài toán. Ngoài ra vì không biết phương sai tổng thể, nên sẽ sử dụng phương sai của mẫu để thay thế.

Nhóm sẽ giải quyết bài toán theo hai hướng, theo cơ sở lý thuyết đã được học và sử dụng hàm **z.test()** được hỗ trợ bởi R để kiểm tra, so sánh kết quả.

5.2.5 Giải quyết bài toán trên R

Vì xu hướng kiểm định của ta là kiểm tra xem giá trị trung bình **Memory_Bandwidth_Value** trên tổng thể của mẫu thứ nhất (hãng Nvidia) có lớn hơn mẫu thứ hai (hãng AMD) hay không, do đó, giả thuyết nhóm đưa ra là:

- Giả thuyết $H_0: \mu_1 \leq \mu_2$
- Giả thuyết $H_1: \mu_1 > \mu_2$

Xác định giá trị **z_score** ứng với $\alpha = 0.05$:

```
1 # Giá trị z_score ứng với alpha = 0.05
2 alpha <- 0.05
3 z_score <- qnorm(1 - alpha)
```

Do đó, miền bác bỏ là: $RR = (1.6449; \infty)$.

Kế tiếp, tính toán các giá trị trung bình, phương sai của hai mẫu.



```
1 # Tính kích thước, giá trị trung bình và phương sai của hai mẫu
2 size_nvidia <- length(nvidia)
3 mean_nvidia <- mean(nvidia)
4 sd_nvidia <- sd(nvidia)
5
6 size_amd <- length(amd)
7 mean_amd <- mean(amd)
8 sd_amd <- sd(amd)
```

Cuối cùng từ các số liệu đã có, nhóm tính giá trị quan sát để rút ra kết luận.

```
1 # Tính giá trị quan sát Z_qs
2 SE <- sqrt(sd_nvidia^2 / size_nvidia + sd_amd^2 / size_amd)
3 Z_stat <- (mean_nvidia - mean_amd) / SE
4 print(paste("Z_stat = ", Z_qs))

1 Z_stat = 2.56752041896274
```

Từ kết quả thu được, ta thấy giá trị quan sát $Z_{\text{stat}} \in (1.6449; \infty)$.

Do đó, có thể bác bỏ giả thuyết H_0 , chấp nhận giả thuyết H_1 , rằng trên tổng thể, giá trị trung bình về thông số GPU Memory_Bandwidth_Value do hãng Nvidia sản xuất lớn hơn do hãng AMD sản xuất.

Sử dụng thêm công cụ `z.test()` được cung cấp bởi R, ta kiểm tra kết quả đã tính toán.

```
1 library(BSDA)
2 z.test(
3   x = nvidia,
4   y = amd,
5   mu = 0,
6   sigma.x = sd_nvidia,
7   sigma.y = sd_amd,
8   alternative = "greater",
9   conf.level = 1 - alpha
10 )
```

```
1 Two-sample z-Test
2
3 data:  nvidia and amd
4 z = 2.5675, p-value = 0.005121
5 alternative hypothesis: true difference in means is greater than 0
6 95 percent confidence interval:
7  6.252307      NA
8 sample estimates:
9 mean of x mean of y
10 170.8046 153.4062
```

5.2.6 Nhận xét

Quan sát kết quả từ lệnh, ta thấy giá trị $p_value = 0.005121 < \alpha = 0.05$ hay tương đương với $z = 2.5675 > z_stat = 1.6449$. Có thể bác bỏ giả thuyết H_0 , chấp nhận giả thuyết H_1 .

Kết quả hoàn toàn trùng khớp với các giá trị đã tính toán.

5.2.7 Kết luận

Nhóm có cơ hội thực hành, tìm hiểu các lệnh của R để thực hiện việc tính toán, giải quyết bài toán kiểm định trung bình tổng thể hai mẫu độc lập, qua đó có cái nhìn sâu sắc hơn về cơ sở lý thuyết đi cùng với thực hành.

5.3 Phân tích phương sai

5.3.1 Mục tiêu

- Nắm được các khái niệm liên quan đến phân tích phương sai ANOVA.
- Biết cách giải một bài toán phân tích phương sai một yếu tố.
- Tìm hiểu và hiện thực các lệnh của ngôn ngữ R để giải quyết bài toán.

5.3.2 Bài toán

Dùng mô hình ANOVA kiểm định trung bình của yếu tố **Memory_Speed** giữa các nhóm có **manufacturer** (hãng) khác nhau với mức ý nghĩa $\alpha = 5\%$.

5.3.3 Điều kiện để thực hiện bài toán Anova

Để thực hiện được mô hình Anova thì phải thỏa mãn 3 yếu tố sau:

- Trong các mẫu có phân phối chuẩn. Có thể sử dụng đồ thị để kiểm định phân phối chuẩn. Ngoài ra còn có thể thực hiện kiểm định bằng hàm **shapiro.test()** để kiểm định giả thuyết.
- Phương sai tổng thể giữa các của các mẫu phải bằng nhau. Có thể sử dụng hàm **levenTest()** để kiểm tra các phương sai của các mẫu có bằng nhau hay không.
- Các mẫu được lấy phải độc lập với nhau.

5.3.3.1 Kiểm tra - xử lý mẫu phân loại:

Dùng hàm **table()** để thống kê số lượng quan sát tương ứng với từng tên trong mẫu **manufacturer**

```
# Lưu kết quả của hàm table() vào biến và in kết quả đó ra  
print(manufacturer_table <- table(my_data$manufacturer))
```

```
##  
##   AMD   ATI  Intel Nvidia  
##   690   65     2    1209
```

Hình 5.1: Kết quả khi thực hiện đoạn code trên

Ta thấy, có một vài loại GPU có số lượng quan sát khá thấp. Do đó, để mô hình ANOVA chính xác hơn, ta sẽ lọc bỏ các quan sát có số lượng không lớn hơn 10.

```
# Trích xuất dữ liệu với số lượng mỗi quan sát theo tên nhiều hơn 10
my_data_filtered <- my_data[my_data$manufacturer %in% names( manufacturer_table [ manufacturer_table > 10]), ]
# Kiểm tra lại thống kê các hãng
table(my_data_filtered$manufacturer)
```

```
##
##   AMD   ATI Nvidia
##  690    65   1209
```

Hình 5.2: Kết quả khi thực hiện đoạn code trên

Sau khi thực thi đoạn chương trình trên, ta còn lại ba hãng sản xuất. Đây sẽ là ba hãng sản xuất cho đầu vào của mô hình ANOVA.

5.3.3.2 Kiểm tra điều kiện 1: Các mẫu phải có phân phối chuẩn.

Để kiểm tra các mẫu có phân phối chuẩn hay không, ta sử dụng phương pháp là sử dụng đồ thị nhưng phương pháp sử dụng đồ thị chỉ mang tính cảm quan nên để chắc chắn nhóm đã đề xuất sử dụng thêm hàm **shapiro.test()** để kiểm định giả thuyết.

- Với phương pháp sử dụng đồ thị, để nhận biết mẫu có phân phối chuẩn thì các điểm quan trắc phải nằm xung quanh đường thẳng thì có thể kết luận mẫu có phân phối chuẩn.
- Với phương pháp sử dụng hàm **shapiro.test()**, để nhận biết mẫu có phân phối chuẩn thì ta có thể đặt các giả thiết để kiểm tra phân phối chuẩn với mức ý nghĩa $\alpha = 5\%$:

$$\begin{cases} H_0 : X \text{ tuân theo phân phối chuẩn} \\ H_1 : X \text{ không tuân theo phân phối chuẩn} \end{cases}$$

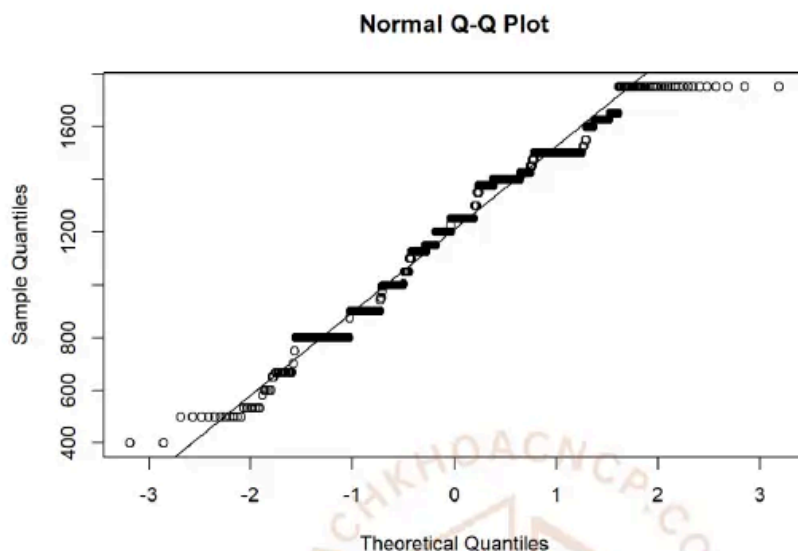
Sau đó cần kiểm tra giá trị **p-value** mà hàm trả về

- p-value** $\leq \alpha$: ta bác bỏ giả thuyết H_0 , tức thừa nhận mẫu không tuân theo phân phối chuẩn.
- p-value** $> \alpha$: ta chưa thể bác bỏ giả thuyết H_0 , tức chưa thể kết luận X không tuân theo phân phối chuẩn. Lúc này ta có thể coi như X tuân theo phân phối chuẩn

1. Kiểm tra mẫu AMD

Sử dụng đồ thị:

```
#Vẽ biểu đồ phân phối chuẩn cho AMD
AMD_data <- subset(my_data_filtered,my_data_filtered$manufacturer=="AMD")
qqnorm(AMD_data$memory_speed_value)
qqline(AMD_data$memory_speed_value)
```



Hình 5.3: Sử dụng đồ thị để kiểm tra phân phối đều của hãng AMD

Sử dụng hàm `shapiro.test()`:

```
#Kiểm tra phân phối chuẩn cho AMD bằng hàm shapiro.test
shapiro.test(AMD_data$memory_speed_value)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  AMD_data$memory_speed_value
## W = 0.97096, p-value = 1.838e-10
```

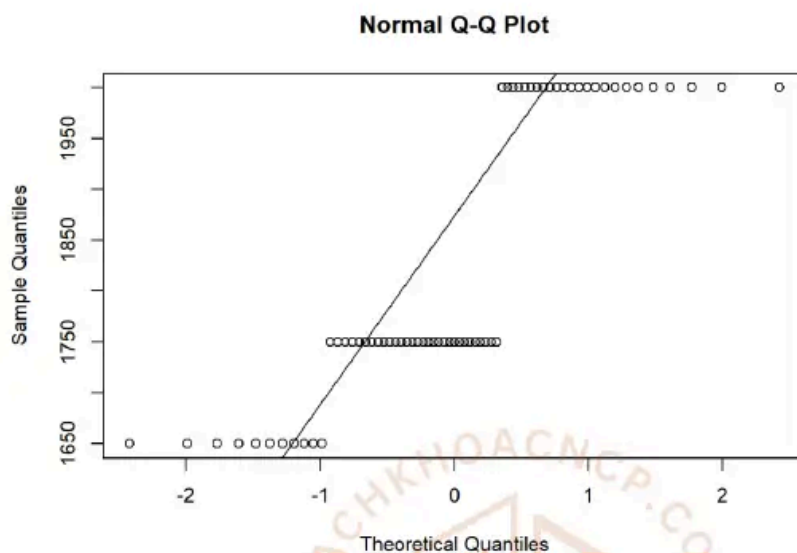
Hình 5.4: Sử dụng hàm `shapiro.test()` để kiểm tra phân phối đều của hãng AMD

Nhận xét: Nhìn vào đồ thị kết hợp với giá trị **p-value** của hàm `shapiro.test()` trả về với **p-value** < 5% nên có thể kết luận mẫu không có phân phối chuẩn

2. Kiểm tra mẫu ATI

Sử dụng đồ thị:

```
#Vẽ biểu đồ phân phối chuẩn cho ATI
ATI_data <- subset(my_data_filtered, my_data_filtered$manufacturer=="ATI")
qqnorm(ATI_data$memory_speed_value)
qqline(ATI_data$memory_speed_value)
```



Hình 5.5: Sử dụng đồ thị để kiểm tra phân phối đều của hãng ATI

Sử dụng hàm `shapiro.test()`:

```
#Kiểm tra phân phối chuẩn cho ATI bằng hàm shapiro.test
shapiro.test(ATI_data$memory_speed_value)
```

```
##
## Shapiro-Wilk normality test
##
## data: ATI_data$memory_speed_value
## W = 0.74895, p-value = 3.343e-09
```

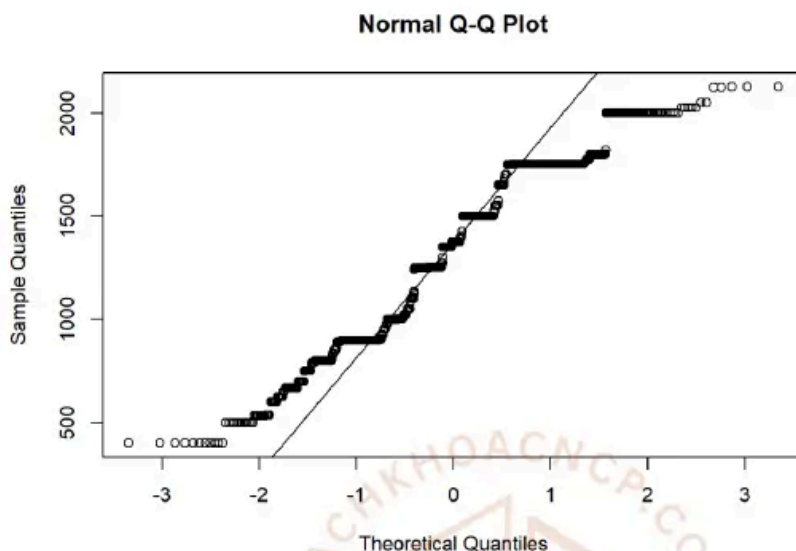
Hình 5.6: Sử dụng hàm `shapiro.test()` để kiểm tra phân phối đều của hãng ATI

Nhận xét: Nhìn vào đồ thị kết hợp với giá trị **p-value** của hàm `shapiro.test()` trả về với **p-value** < 5% nên có thể kết luận mẫu không có phân phối chuẩn

3. Kiểm tra mẫu Nvidia

Sử dụng đồ thị:

```
#Vẽ biểu đồ phân phối chuẩn cho Nvidia
Nvidia_data <- subset(my_data_filtered, my_data_filtered$manufacturer=="Nvidia")
qqnorm(Nvidia_data$memory_speed_value)
qqline(Nvidia_data$memory_speed_value)
```



Hình 5.7: Sử dụng đồ thị để kiểm tra phân phối đều của hãng Nvidia

Sử dụng hàm `shapiro.test()`:

```
#Kiểm tra phân phối chuẩn cho Nvidia bằng hàm shapiro.test
shapiro.test(Nvidia_data$memory_speed_value)
```

```
##
## Shapiro-Wilk normality test
##
## data:  Nvidia_data$memory_speed_value
## W = 0.95021, p-value < 2.2e-16
```

Hình 5.8: Sử dụng hàm `shapiro.test()` để kiểm tra phân phối đều của hãng Nvidia

Nhận xét: Nhìn vào đồ thị kết hợp với giá trị **p-value** của hàm `shapiro.test()` trả về với **p-value** < 5% nên có thể kết luận mẫu không có phân phối chuẩn. Trên lý thuyết, mẫu này không thỏa giả thiết có thể áp dụng mô hình ANOVA, tuy nhiên với giới hạn về mặt dữ liệu, ta giả sử mẫu thỏa giả thiết và tiếp tục thực hiện mô hình.

5.3.3.3 Kiểm tra điều kiện 2: Các mẫu có phương sai đồng thời bằng nhau

Ngoài điều kiện các mẫu có phân phối chuẩn thì chúng cũng cần phải đáp ứng được yêu cầu các phương sai của các mẫu bằng nhau. Do đó, ta cần kiểm định thêm về sự tương đồng của phương sai các nhóm thông qua hàm `leveneTest()`. Với giả thuyết:

$$\begin{cases} H_0 : \text{phương sai của các mẫu đồng thời bằng nhau} \\ H_1 : \text{tồn tại một phương sai khác với các phương sai còn lại} \end{cases}$$

Sau đó cần kiểm tra giá trị **p-value** mà hàm trả về

- **p-value** $\leq \alpha$: ta bác bỏ giả thuyết H_0 , tức thừa nhận có một phương sai khác với các phương sai còn lại.
- **p-value** $> \alpha$: ta chưa thể bác bỏ giả thuyết H_0 , tức chưa thể kết luận tồn tại một phương sai khác với các phương sai còn lại. Lúc này t có thể coi như các phương sai bằng nhau.

1. Kiểm tra phương sai

```
# sử dụng hàm leveneTest() để thực hiện kiểm tra điều kiện phương sai bằng nhau giữa các mẫu
library("car") # input thư viện car để sử dụng hàm leveneTest()

## Loading required package: carData

leveneTest(memory_speed_value~as.factor(manufacturer),my_data_filtered)

## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value    Pr(>F)
## group  2  81.888 < 2.2e-16 ***
##      1961
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Hình 5.9: Sử dụng hàm `leveneTest()` để kiểm tra các phương sai

Nhận xét: Nhận xét: p-value của kiểm định rất nhỏ so với mức ý nghĩa $\alpha = 5\%$. Điều này có nghĩa ta bác bỏ giả thuyết về sự bằng nhau giữa phương sai các nhóm. Nói cách khác, có sự khác biệt về phương sai giữa các hãng sản xuất. Trên lý thuyết, mẫu này không thoả giả thiết có thể áp dụng mô hình ANOVA, tuy nhiên với giới hạn về mặt dữ liệu, ta giả sử mẫu thoả giả thiết và tiếp tục thực hiện mô hình.

5.3.4 Phân tích phương sai

5.3.4.1 Tiến hành phân tích phương sai

Dùng hàm `aov()` kết hợp `summary()` với những tham số phù hợp để tiến hành áp dụng mô hình phân tích phương sai một yếu tố cho mẫu `memory_speed_value`, phân loại theo `manufacturer` đã lọc.

```
# Lưu kết quả của mô hình ANOVA vào biến (để sử dụng lại cho sau này)
anova_result <- aov (memory_speed_value ~ manufacturer, data = my_data_filtered)
# Hiển thị kết quả chi tiết của mô hình ANOVA
summary(anova_result)

##              Df    Sum Sq Mean Sq F value Pr(>F)
## manufacturer  2  25846920 12923460   96.77 <2e-16 ***
## Residuals    1961  261896870   133553
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Hình 5.10: Kết quả thực thi đoạn chương trình trên

Từ kết quả thu được ở trên, ta thống kê các giá trị đặc trưng của mô hình ANOVA:

- Bậc tự do $DfB = k - 1 = 3$
- Bậc tự do $DfW = N - k = 1961$
- $SSB = 25846920$
- $SSW = 261896870$
- $MSB = 12923460$
- $MSW = 133553$
- Kiểm định thống kê $F = 96.77$
- p-value $< 2.10^{-16}$

Nhận xét:

Giá trị thống kê kiểm định F biểu thị tỷ lệ bình phương trung bình giữa các nhóm với bình phương trung bình trong mỗi nhóm. Giá trị F khá cao, cho thấy trung bình của nhóm không phải tất cả đều bằng nhau.

Hơn nữa, **p-value** thấp hơn rất nhiều so với mức ý nghĩa $\alpha = 5\%$, cho thấy rõ ràng sự khác biệt về trung bình giữa các nhóm được quan sát là có ý nghĩa thống kê.

Tóm lại, kết quả mô hình ANOVA cho bài toán này dẫn đến việc bác bỏ giả thuyết không H_0 – giả thuyết cho rằng giá trị **memory_speed_value** trung bình của các nhóm đều bằng nhau. Nói cách khác, ta có thể kết luận giá trị trung bình **memory_speed_value** của các hãng khác nhau không cùng bằng nhau.

5.3.4.2 Phân tích sâu

Ta chỉ thực hiện phân tích sâu khi có sự khác biệt về trung bình của bài toán ANOVA. Do giả thuyết không H_0 đã được bác bỏ, ta có nhu cầu kiểm định giả thuyết về trung bình **memory_speed_value** giữa từng nhóm.

Sử dụng hàm **TukeyHSD()** để thực hiện phân tích sâu giữa các nhóm. Với n nhóm thì ta sẽ có C_n^2 cặp so sánh bội. Đặt giả thuyết:

$$\begin{cases} H_0 : \text{trung bình cặp so sánh bội bằng nhau} \\ H_1 : \text{trung bình cặp so sánh bội nhóm khác nhau} \end{cases}$$

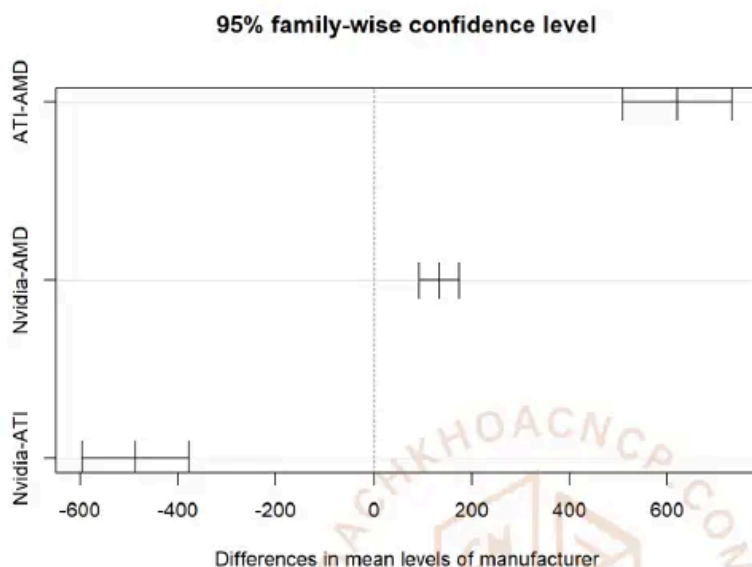
```
# Dùng hàm TukeyHSD() để thực hiện so sánh bội giữa các nhóm
TukeyHSD(anova_result)

## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = memory_speed_value ~ manufacturer, data = my_data_filtered)
##
## $manufacturer
##          diff      lwr      upr p adj
## ATI-AMD    620.3890 509.17706 731.6009  0
## Nvidia-AMD 133.5304  92.63412 174.4267  0
## Nvidia-ATI -486.8586 -595.99604 -377.7211  0
```

Hình 5.11: Kết quả thực thi quá trình trên.

Đồ thị mô tả phân tích sâu:

```
# vẽ biểu đồ cho việc so sánh bội giữa các nhóm
plot(TukeyHSD(anova_result))
```



Hình 5.12: Đồ thị mô tả phân tích sâu.

Nhận xét: Vì ta có 3 nhóm nên sẽ có C_3^2 cặp cần so sánh:

- Xét cặp so sánh bội ATI và AMD thì ta thấy **p-value** $< 5\%$ nên ta có thể bác bỏ H_0 từ đó có thể kết luận $\mu_1 \neq \mu_2$ và vì $diff > 0 (\bar{x}_1 > \bar{x}_2)$ nên $\mu_1 > \mu_2$
- Xét cặp so sánh bội Nvidia và AMD thì ta thấy **p-value** $< 5\%$ nên ta có thể bác bỏ H_0 từ đó có thể kết luận $\mu_1 \neq \mu_2$ và vì $diff > 0 (\bar{x}_1 > \bar{x}_2)$ nên $\mu_1 > \mu_2$
- Xét cặp so sánh bội Nvidia và ATI thì ta thấy **p-value** $< 5\%$ nên ta có thể bác bỏ H_0 từ đó có thể kết luận $\mu_1 \neq \mu_2$ và vì $diff < 0 (\bar{x}_1 < \bar{x}_2)$ nên $\mu_1 < \mu_2$

⇒ Có thể rút ra kết luận tổng về sự khác nhau giữa ba hãng sản xuất về tốc độ của GPUs với tốc độ trung bình được xếp theo thứ tự $AMD < Nvidia < ATI$

5.3.5 Kết luận

Trong bài toán này, nhóm đã thành công trong việc nghiên cứu các lý thuyết về phân tích phương sai ANOVA, tìm hiểu các lệnh của **R** để thực hiện việc tính toán, áp dụng mô hình ANOVA cho một yếu tố cụ thể, đồng thời so sánh bội được giả thuyết về trung bình giữa các nhóm yếu tố, đưa ra được kết luận cuối cùng trong bài toán.

Tuy nhiên, kết quả nhóm trình bày có thể chưa thật sự chính xác. Nguyên nhân có thể đến từ nhiều yếu tố:

- Ảnh hưởng của việc loại bỏ các quan sát khuyết trong quá trình tiền xử lý số liệu.



- Tổng thể chưa thoả giả thiết của mô hình: phân phối chưa thật sự chuẩn, phương sai của các nhóm không bằng nhau.
- Sự chênh lệch lớn về số lượng quan sát giữa các nhóm.

Chính vì những hạn chế trên, nhìn chung kết quả mà nhóm tìm được có thể có sai khác so với kết quả chính xác.



5.4 Hồi quy tuyến tính

5.4.1 Mục tiêu

- Nắm được các khái niệm liên quan đến mô hình hồi quy tuyến tính.
- Biết cách giải một bài toán hồi quy tuyến tính đơn, hồi quy tuyến tính bội.
- Tìm hiểu và hiện thực các lệnh của ngôn ngữ R để giải quyết bài toán, biết cách đọc, giải thích, vẽ các biểu đồ minh họa cho kết quả.

5.4.2 Bài toán

Xây dựng mô hình hồi quy để đánh giá các yếu tố ảnh hưởng đến **release_price** (giá phát hành) với mức ý nghĩa $\alpha = 5\%$

Xây dựng mô hình hồi quy trong đó:

- Biến phụ thuộc: **release_price**
- Biến độc lập: **number_of_pixels**, **core_speed_value**, **memory_value**, **memory_bandwidth_value**, **memory_speed_value**, **manufacturer**, **max_power_value**

5.4.3 Xây dựng mô hình hồi quy tuyến tính

5.4.3.1 Xử lý dữ liệu

Trích ra các mẫu định lượng trong tập dữ liệu, đồng thời lọc lấy những quan sát có yếu tố **release_price** trong tập dữ liệu.

```
# Trích các biến phụ thuộc và các biến độc lập
samples <- my_data[,c("release_price", "number_of_pixels",
"core_speed_value", "memory_value", "memory_bandwidth_value",
"memory_speed_value", "manufacturer", "max_power_value")]
head(samples,10) #In ra 10 dòng đầu của bộ data samples
```

	release_price	number_of_pixels	core_speed_value	memory_value
## 1	371.5624	2073600	870	2048
## 2	371.5624	2073600	650	6144
## 3	371.5624	4096000	706	12288
## 4	371.5624	1440000	1050	3072
## 5	371.5624	3686400	837	6144
## 6	2999.0000	9216000	705	12288
## 7	2999.0000	8294400	705	12288
## 8	1099.0000	8294400	1140	12288
## 9	1998.0000	6220800	1000	24576
## 10	999.0000	8294400	1000	12288

	memory_bandwidth_value	memory_speed_value	manufacturer	max_power_value
## 1	134.4	1050	AMD	190
## 2	177.6	925	Nvidia	250
## 3	288.4	1502	Nvidia	245
## 4	57.6	900	AMD	150
## 5	288.4	1502	Nvidia	250
## 6	672.0	1750	Nvidia	375
## 7	672.0	1750	Nvidia	375
## 8	336.6	1753	Nvidia	250
## 9	673.2	1753	Nvidia	450
## 10	336.6	1753	Nvidia	250

Hình 5.13: Trích xuất biến phụ thuộc và biến độc lập.

5.4.3.2 Tổng quan mô hình

Ta có mô hình tổng quát theo những dữ liệu đã được trích xuất ở trên theo cấu trúc sau:
Gọi $Y, X_1, X_2, X_3, X_4, X_5, X_6, X_7$ lần lượt là các biến `release_price`, `number_of_pixels`, `core_speed_value`, `memory_value`, `memory_bandwidth_value`, `memory_speed_value`, `manufacturer`, `max_power_value`

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \beta_5 X_5 + \beta_6 X_6 + \beta_7 X_7 + \epsilon$$

Sau khi thực hiện mô hình hồi quy tuyến tính ta sẽ được phương trình hồi quy:

$$\widehat{Y} = \widehat{\beta}_1 X_1 + \widehat{\beta}_2 X_2 + \widehat{\beta}_3 X_3 + \widehat{\beta}_4 X_4 + \widehat{\beta}_5 X_5 + \widehat{\beta}_6 X_6 + \widehat{\beta}_7 X_7$$

5.4.3.3 Xây dựng mô hình

Dùng hàm `lm()` để tiến hành áp dụng mô hình hồi quy tuyến tính cho bộ dữ liệu đã xử lý, với `release_price` là biến phụ thuộc, các biến còn lại là các biến độc lập. Với giả thuyết:

$$\begin{cases} H_0 : \text{biến độc lập không có ý nghĩa cho mô hình} \\ H_1 : \text{biến độc lập có ý nghĩa cho mô hình} \end{cases}$$

Ta xét **p-value** và so sánh với mức ý nghĩa $\alpha = 5\%$ để kết luận giả thuyết cho từng biến.

```
# Dữ liệu thể hiện các biến còn lại trong samples
lm_model <- lm( release_price ~ ., samples )
summary(lm_model)
```

```
##
## Call:
## lm(formula = release_price ~ ., data = samples)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -710.0    -76.3       5.7     67.6   13481.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    4.246e+02   3.674e+01  11.560 < 2e-16 ***
## number_of_pixels -5.390e-06   1.249e-05  -0.432  0.666067
## core_speed_value -1.870e-01   5.582e-02  -3.350  0.000824 ***
## memory_value      5.782e-02   5.833e-03   9.912 < 2e-16 ***
## memory_bandwidth_value -1.021e+00  2.170e-01  -4.703  2.74e-06 ***
## memory_speed_value  -7.046e-02  3.402e-02  -2.071  0.038504 *
## manufacturerATI    -1.553e+02   4.940e+01  -3.143  0.001695 **
## manufacturerIntel  1.186e+01   2.440e+02   0.049  0.961243
## manufacturerNvidia  4.502e+01   1.737e+01   2.592  0.009626 **
## max_power_value    1.338e+00   1.865e-01   7.176  1.02e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 342.3 on 1956 degrees of freedom
## Multiple R-squared:  0.1221, Adjusted R-squared:  0.1181
## F-statistic: 30.24 on 9 and 1956 DF, p-value: < 2.2e-16
```

Hình 5.14: Kết quả thực thi đoạn chương trình trên.

Đánh giá p-value

- **number_of_pixels**: $p\text{-value} = 0.666067 > 5\% \Rightarrow$ Chấp nhận H_0 - Biến không có ý nghĩa với

mô hình.

- **core_speed_value**: **p-value** = $0.000824 < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **memory_value**: **p-value** = $2 * 10^{-16} < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **memory_bandwidth_value**: **p-value** = $2.74 * 10^{-6} < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **memory_speed_value**: **p-value** = $0.038504 < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **manufacturerATI**: **p-value** = $0.001695 < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **manufacturerIntel**: **p-value** = $0.961243 > 5\% \Rightarrow$ Chấp nhận H_0 - Biến không có ý nghĩa với mô hình.
- **manufacturerNvidia**: **p-value** = $0.009626 < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.
- **max_power_value**: **p-value** = $1.02 * 10^{-12} < 5\% \Rightarrow$ Bác bỏ H_0 - Thừa nhận biến có ý nghĩa với mô hình.

Nhận xét

Dựa vào cột **Estimate**, ta có được các hệ số $a, b_1, b_2, \dots$ của mô hình, từ đó xác định được phương trình hồi quy tuyến tính mẫu:

Lúc này, phương trình hồi quy đa biến có dạng:

$$\begin{aligned}\widehat{Y} = & 424.6 - 5.39 * 10^{-6}X_1 - 0.187X_2 + 0.05782X_3 - 1.021X_4 - 0.07046X_5 - 155.3X_6ATI + \\ & 11.86X_6Intel + 45.02X_6Nvidia + 1.338X_7\end{aligned}$$

Hệ số hiệu chỉnh: $R^2 = 11.81\%$

Xây dựng mô hình tối ưu

Do các biến **number_of_pixels** và **manufacturerIntel** không có ý nghĩa cho mô hình, ta sẽ loại bỏ các biến này, đồng thời giữ lại những biến còn lại.

```
# Dấu . thể hiện các biến còn lại trong samples
lm_model2 <- lm( release_price ~ ., samples )
summary(lm_model)

##
## Call:
## lm(formula = release_price ~ ., data = samples)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -717.7   -75.8     6.1    68.4  13486.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    4.226e+02  3.642e+01  11.602 < 2e-16 ***
## core_speed_value -1.897e-01  5.547e-02  -3.420 0.000638 ***
## memory_value     5.689e-02  5.416e-03  10.504 < 2e-16 ***
## memory_bandwidth_value -1.076e+00  1.753e-01  -6.137 1.02e-09 ***
## memory_speed_value  -6.914e-02  3.389e-02  -2.040 0.041447 *
## manufacturerATI    -1.524e+02  4.895e+01  -3.113 0.001876 **
## manufacturerNvidia  4.336e+01  1.694e+01  2.559 0.010562 *
## max_power_value    1.364e+00  1.767e-01  7.718 1.87e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 342.3 on 1956 degrees of freedom
## Multiple R-squared:  0.1221, Adjusted R-squared:  0.1189
## F-statistic: 38.85 on 7 and 1956 DF, p-value: < 2.2e-16
```

Hình 5.15: Kết quả thực thi đoạn chương trình trên.

Lúc này, phương trình hồi quy đa biến tối ưu có dạng:

$$\hat{Y} = 422.6 - 0.1897X_2 + 0.05609X_3 - 1.076X_4 - 0.06914X_5 - 152.4X_6ATI + 43.36X_6Nvidia + 1.364X_7$$

Hệ số hiệu chỉnh: $R^2 = 11.89\%$

Đánh giá: Đây là một tỉ lệ tương đối thấp, cho thấy các biến độc lập đưa vào phân tích hồi quy giải thích được 11.89% sự biến thiên của biến phụ thuộc, phần còn lại có thể được giải thích bởi phần dư gồm các biến độc lập ngoài mô hình và sai số ngẫu nhiên.

Ngoài ra, sau khi thực hiện loại bỏ các biến không ảnh hưởng đến phương trình hồi quy tuyến tính thì ta thấy hệ số hiệu chỉnh tăng lên, từ đó có thể giải thích được phương trình càng tối ưu thì hệ số hiệu chỉnh càng cao. Từ đó hiệu quả của mô hình càng lớn, để kiểm chứng cho nhận định trên ta thực hiện việc so sánh hiệu quả 2 mô hình.

So sánh hiệu quả hai mô hình

Để so sánh hiệu quả hai mô hình ta sử dụng hàm **anova()** để kiểm định hiệu quả của hai mô hình. Với giả thuyết:

$$\begin{cases} H_0 : \text{mô hình 2 hiệu quả hơn mô hình 1} \\ H_1 : \text{mô hình 1 hiệu quả hơn mô hình 2} \end{cases}$$

```
# thực hiện so sánh hai mô hình
anova(lm_model2, lm_model)

## Analysis of Variance Table
##
## Model 1: release_price ~ release_price + core_speed_value + memory_value +
##   memory_bandwidth_value + memory_speed_value + manufacturer +
##   max_power_value
## Model 2: release_price ~ release_price + number_of_pixels + core_speed_value +
##   memory_value + memory_bandwidth_value + memory_speed_value +
##   manufacturer + max_power_value
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1    1957 229153659
## 2    1956 229131836  1      21823 0.1863 0.6661
```

Hình 5.16: Kết quả thực thi đoạn chương trình trên.

Nhận xét:

Vì $p\text{-value} = 0.6661 > 5\%$ nên chưa thể bác bỏ H_0 , đồng nghĩa với việc chấp nhận giả thuyết mô hình 2 hiệu quả hơn so với mô hình 1.

5.4.3.4 Ước lượng khoảng tin cậy cho các hệ số

Dùng hàm `confint()` để tìm các khoảng ước lượng cho các hệ số $\beta_1, \beta_2, \beta_3, \dots, \beta_n$ của mô hình hồi quy tuyến tính cho giá trị `release_price` của tổng thể các GPU với độ tin cậy 95%.

```
# Tìm khoảng ước lượng cho các hệ số
confint(lm_model2, level = 0.95)

##               2.5 %           97.5 %
## (Intercept)    351.16176722 494.031950772
## core_speed_value -0.29849606 -0.080938002
## memory_value     0.04626534  0.067508102
## memory_bandwidth_value -1.41966468 -0.732018756
## memory_speed_value -0.13560139 -0.002683769
## manufacturerATI  -248.41795830 -56.406081557
## manufacturerNvidia 10.13500724 76.592957768
## max_power_value   1.01740684  1.710641569
```

Hình 5.17: Kết quả thực thi đoạn chương trình trên.

Từ kết quả trên, ta xác định được các khoảng tin cậy cho hệ số chặn β_0 cũng như các hệ số của các biến độc lập trong mô hình tổng thể với độ tin cậy 95%:

5.4.3.5 Kiểm tra giả thuyết

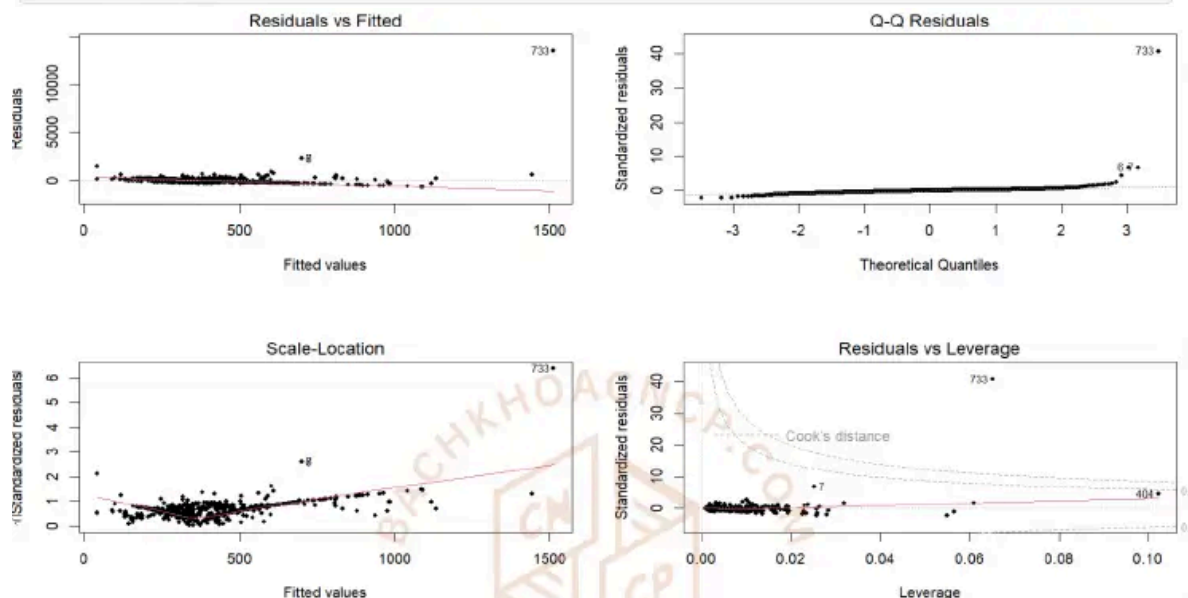
Từ kết quả có được về mô hình hồi quy tuyến tính đã xây dựng, ta cần kiểm định lại các giả thiết của mô hình:

1. Tính tuyến tính của dữ liệu.
2. Phần dư có phân phối chuẩn.
3. Phần dư có trung bình bằng 0.
4. Phần dư có phương sai không đổi.
5. Sai số ngẫu nhiên có phân phối chuẩn.

6. Sai số ngẫu nhiên có kỳ vọng tại mỗi giá trị bằng 0.

Dùng hàm `plot()` vẽ biểu đồ phần dư để kiểm tra các giả thiết về phần dư trên cho mô hình đã xây dựng:

```
par ( mfrow = c(2, 2))
plot(lm_model12, pch=20)
```



Hình 5.18: Kết quả thực thi đoạn chương trình trên.

Ý nghĩa của các biểu đồ:

- **Biểu đồ Residuals vs Fitted:** Dùng để kiểm tra giả thiết tuyến tính của dữ liệu và giả thiết phần dư có trung bình bằng 0. Trục tung biểu thị giá trị của phần dư, trục hoành biểu thị giá trị tiên lượng $\hat{y}_i$ của biến phụ thuộc. Nếu đường màu đỏ trên biểu đồ càng có dạng một đường thẳng nằm ngang, điều đó càng chứng tỏ tính tuyến tính của dữ liệu càng cao. Mặc khác, giả thiết phần dư có trung bình bằng 0 thỏa mãn nếu đường màu đỏ gần với đường nét đứt nằm ngang (ứng với phần dư bằng 0) trên biểu đồ.
- **Biểu đồ Normal Q-Q:** Dùng để kiểm tra giả thiết phần dư có phân phối chuẩn. Nếu các điểm thẳng dư có xu hướng phân bố trên một đường thẳng thì điều kiện về phân phối chuẩn của phần dư được thỏa.
- **Biểu đồ Scale - Location:** Dùng để kiểm định giả thiết phương sai của phần dư là không đổi. Trục tung là căn bậc hai của phần dư (đã được chuẩn hóa), trục hoành là giá trị tiên lượng $\hat{y}_i$ của các biến phụ thuộc. Nếu đường màu đỏ trên đồ thị là đường thẳng nằm ngang và các điểm thẳng dư phân tán đều xung quanh đường thẳng này thì giả thiết về phương sai của phần dư được thỏa.
- **Biểu đồ Residuals vs Leverage:** mặc dù không dùng để kiểm định giả thiết cho mô hình, tuy nhiên biểu đồ này giúp xác định những điểm outliers (các phần tử bất thường có thể là nguyên nhân gây ra sự vi phạm các giả thiết hay làm sai lệch kết quả dự báo của mô hình). Những điểm

outliers sẽ cách xa đường màu đỏ trên biểu đồ.

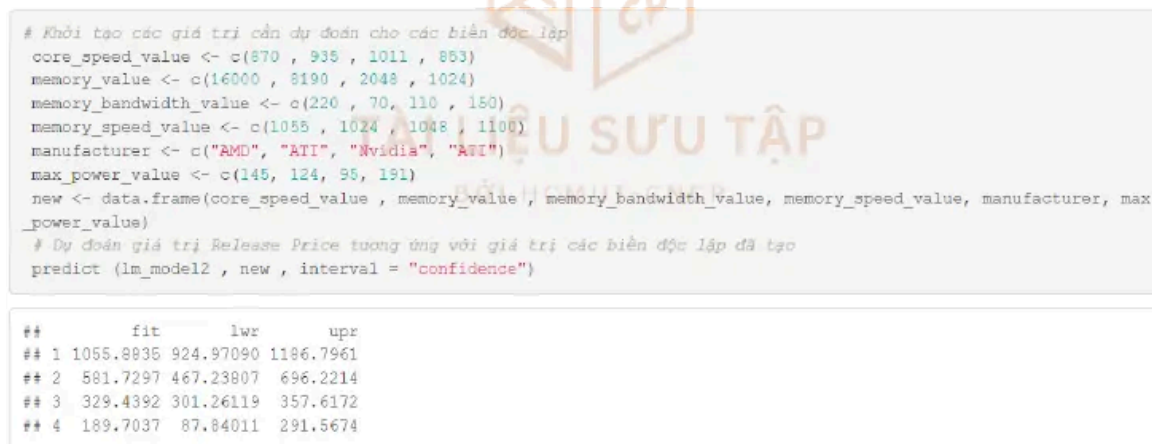
Nhận xét:

- **Biểu đồ Residuals vs Fitted** cho thấy giả thiết về tính tuyến tính của dữ liệu là có thể chấp nhận được. Tuy nhiên giả thiết về trung bình của phần dư có thể coi là thỏa mãn.
- **Biểu đồ Normal Q-Q** cho thấy giả thiết về phần dư có phân phối chuẩn được thỏa mãn.
- **Biểu đồ Scale - Location** đường màu đỏ có độ dốc, không thẳng, các điểm thẳng dư phân tán không đều xung quanh đường thẳng này. Do đó, giả thiết về tính đồng nhất của phương sai đối với mô hình này bị vi phạm.
- **Biểu đồ Residuals vs Leverage** chỉ ra quan sát thứ 733 có thể là điểm outliers, gây ảnh hưởng đến mô hình đã xây dựng. Vì vậy cần loại bỏ đi để mô hình chuẩn hơn.

5.4.3.6 Dự đoán

Vì lý do hệ số hiệu chỉnh R^2 chỉ có 11.89% nên việc dự đoán là không hiệu quả. Do đó, cần phải cân nhắc để thực hiện dự đoán cho mô hình hồi quy tuyến tính này. Giả sử ta có hệ số hiệu chỉnh cao và thực hiện quá trình dự đoán cho mô hình hồi quy tuyến tính đã xây dựng ở trước.

Từ mô hình hồi quy mẫu đã xây dựng, ta có thể thực hiện việc dự đoán giá trị **release_price** thông qua giá trị của các biến độc lập trong mô hình:



Hình 5.18: Kết quả thực thi đoạn chương trình trên.

Nhận xét: Kết quả cho ta các dự đoán và khoảng ước lượng của dự đoán về giá ra mắt GPU tương ứng với các thông số độc lập trong mô hình.

Ví dụ, với một mẫu GPU có **core_speed_value** là 870[MHz], **memory_value** là 16000[MB], **memory_bandwidth_value** là 220[GB/sec], **memory_speed_value** là 1055[MHz] thì giá ra mắt dự đoán cho mẫu GPU này là khoảng \$1055.8835 và khoảng ước lượng cho giá trị ra mắt thật sự là (924.97090; 1186.7961).



5.4.4 Kết luận

Nhóm đã thành công trong việc nghiên cứu các lý thuyết về mô hình hồi quy tuyến tính đơn, hồi quy tuyến tính bội; tìm hiểu các lệnh của R để thực hiện việc tính toán, xây dựng mô hình, kiểm định giả thiết cho mô hình. Hơn nữa, từ mô hình đã xây dựng, nhóm cũng đã đưa ra được những dự đoán cho biến phụ thuộc.

Tuy nhiên, kết quả nhóm trình bày có thể chưa thật sự chính xác. Nguyên nhân có thể đến từ nhiều yếu tố:

- Tỷ lệ khuyết dữ liệu lớn, ảnh hưởng của việc loại bỏ các quan sát khuyết trong quá trình tiền xử lý số liệu.
- Sự hạn chế về mặt dữ liệu, các mẫu dữ liệu không thỏa mãn các giả thiết cần của mô hình, dẫn đến kết quả chưa thật sự có ý nghĩa lớn về mặt thống kê.

Chính vì những hạn chế trên, nhìn chung kết quả mà nhóm tìm được có thể có sai khác so với kết quả chính xác.





6 Thảo luận và mở rộng





7 Nguồn dữ liệu và nguồn code

- Nguồn file Intel_CPU.csv : <https://www.kaggle.com/datasets/iliassekkaf/computerparts/>
- Nguồn code:





Tài liệu

